

ODD Protocol MAPE-K ELV CRP model

This protocol describes the structure and design of the MAPE-K CRP model, based on the (Extended) MAPE-K CRP model (MAPE-K ELV CRP). The main objective of this document is to build an operational agent-based model to control a managed system, which is agnostic to its application. The MAPE-K model was developed many years ago [1] and is recently extended to facilitate learning [2]. In my research I consider adding the Complex Responsive Process (CRP) patterns between agents to this extended MAPE-K model. In this model, the knowledge base is shared between agents with the mechanisms of complex responsive processes and trust groups. For this purpose, from now on I will refer to this model as the *MAPE-K ELV CRP* model.

Background of the ODD Protocol

Describing and Agent Based Model (ABM) in Multi-Agent Systems (MAS) in a conventional manner for software development raises several pragmatic issues: Duplication of functionality will make a functional design inefficient, agent specific characteristics can be described insufficiently, while science requires the ability to replicate outcomes. A common language for ABM should overcome these drawbacks. For that reason, a de-facto standard has been developed: the ODD protocol (Overview, Design, Details) [3].

An ODD typically is written for human understanding and will not include technical coding, although logical algorithms would be allowed to improve the understanding of the model behavior. Also, the protocol is agnostic and would replace a functional design in a development cycle for the development of MAS solutions. Since the launch of the protocol, it has been adopted widely in scientific model development, especially in ecology and in social and social-ecological sciences [4].

When an ODD is applied, the several weak points should be covered, like the lack of guidance, need for a rationale, length of the document, lack of hierarchy, inefficient re-use of the building blocks of the model, missing link to pattern-oriented development and ambiguity [3].

The ODD model consists of 3 categories and 7 elements. The Design concepts are split into 11 concepts. Based on this structure, a harmonized way of developing ABM models in MAS environments should be developed. In this model description this structure is applied, while a serious effort is taken to overcome the weak points to achieve a transparent solution.

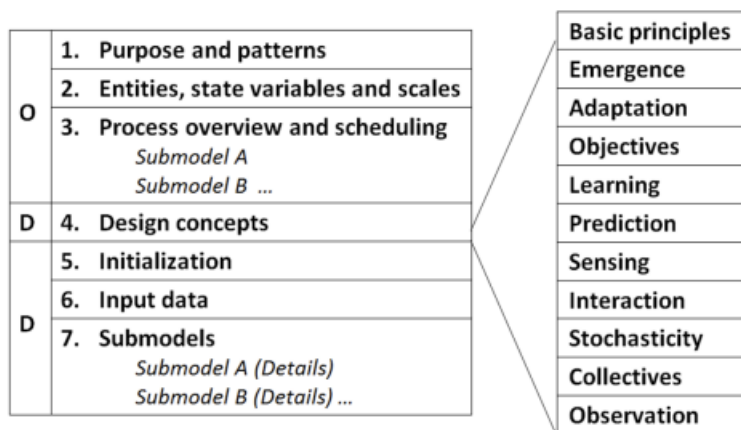


Figure 1: Structure of the ODD model [3]

In this model, the UML class diagrams are added for each sub model, and the Entity Relation Diagram (ERD) is depicted in the input data element.

Overview

In this chapter the layout of the ABM model is described, including its purpose, entities, state variables and scales. As this is developed for generic use, the split between the operational process (managed system), its control loop characteristics (MAPE), the knowledge base (KB), learning capabilities (ELV) and its agent interaction (CRP) is represented as sub models. Each sub model will be defined in more detail later in this protocol.

The Sterman Beer Game

*The **Beer Game**, developed by Jay Forrester and John Sterman the 1960s, is a classic simulation exercise used to teach principles of systems thinking, supply chain management, and the bullwhip effect. The game simulates the dynamics of a multi-tiered supply chain, typically involving four roles: retailer, wholesaler, distributor, and factory. Each participant's goal is to meet customer demand for beer while minimizing costs like inventory and backorders.*

In the game, players place orders based on perceived demand, but they only see their immediate supplier and customer, not the entire chain. Demand at the retail level is usually stable or follows a simple pattern, but due to delays in order fulfillment, communication gaps, and individual decision-making biases, participants often create amplified fluctuations upstream. This phenomenon—the bullwhip effect—results in excessive inventory, stockouts, and inefficiencies, even when demand is steady.

The Beer Game illustrates key insights:

- *Delays in information and material flow distort decision-making.*
- *Local optimization (focusing only on one's own inventory) leads to system-wide instability.*
- *Lack of coordination and misaligned incentives worsen inefficiencies.*
- *Overreaction to perceived shortages (e.g., hoarding) exacerbates volatility.*

Sterman's work highlights how human behavior and structural factors (like lead times and order batching) interact to create unintended consequences. The game demonstrates that even rational actors can generate chaotic outcomes in complex systems. It remains a powerful tool for teaching supply chain dynamics, organizational behavior, and the value of information sharing and collaboration.

Purpose

The purpose of the model is directly related to the main objective of the managed process, as this will be the actual constructor in its environment. Decisions that are taken in the MAPE-K control loop are applied in the managed process and are driven by the purpose of this process. It can be stated that MAPE-K is context dependent on the managed process, irrespective of what kind of process this will be. Although MAPE-K originally stems from the area of cybernetics it could be used in all different fields as well.

For this model the generic purpose will be achieved by:

1. Develop a model to simulate a managed process.
 - This managed process is the operational environment which requires controls by decision taking. Logic will be moved out of the managed process to the MAPE-K control loop. As a result, the managed process will become a pure constructor for information processing and/or physical creation.
2. Add a MAPE-K control cycle to a standard process simulation to allocate decision making.
 - The MAPE-K control cycle will include the logic stored in the Knowledge Base (K) where context data is stored and monitored (Monitor), analyzed and interpreted with policies (Analyze), translated into behavioral change (Plan) and applied for execution (Execute).
3. Implement an internalized learning process to update the agents' Knowledge Base
 - The Knowledge Base contains both contextual information (externalized) and policies (Internalized). This Knowledge Base will be imposed by the modeler when the simulation commences. In each MAPE-K loop the context information is updated. As a learning cycle is added to update the agents Knowledge Base, the internalized policies and rules become dynamic. This learning capability exists of three elements: Evaluating, Learning, and Validating (to update the Knowledge Base). This learning (double) loop on top of MAPE-K is called the MAPE-K ELV model.
4. Add interfacing of agent specific Knowledge Base data within the agent population.
 - Updating the Knowledge Base in the MAPE-K ELV is mainly internalized. However, the learning capabilities of other agents could be valuable to speed up the improvement of a Knowledge Base. Successful strategies can be shared among agents by updating Knowledge Base entity records like specific policies of process patterns. Several ABM techniques are available to facilitate this data exchange.
5. Add social bonding between agent cliques.
 - Create groups of agents by adding trust levels. Grouping of agents can be defined in different ways, i.e., imposed by the modeler, self-organized by agents creating formalized groups or even informal group definition (like influencers). When an agent is part of a group, the trust level between agents improves. This could result in free sharing of Knowledge Base data between group agents, called Trust Groups.

Purpose of an ABM model for the Beer Game

The following purposes for an ABM model for the Beer Game are defined:

1. *Develop a model to simulate a full, multi-echelon, multi-cycle simulation run*
2. *Add a MAPE-K control cycle to the BPMN Beer Game process to allocate agent specific decision making*
3. *Implement an internalized learning process to update the agents Knowledge Base of ordering, forecasting and stock management decisions and other context specific characteristics*
4. *Add bi-directional interaction with other agents by sharing information. This can be done in direct verbal or non-verbal communication during simulation.*
5. *Add trust levels and define agent cliques to vary the value of information shared by agents.*

Entities, State variables and scales

What is the ABM model about? After defining the purpose of the model, we must gain insight into the building blocks and the state space of the model. This is achieved by listing all entities of the model, the states of the variables and the scales of the model.

Entities

The different entities describe the characteristics of the model. So, what objects are described in their state space and what dynamic characteristics are stored in the workspace. Contrary to contemporary model development, the processing of data is loosely coupled, where the data of the model does not include its predefined relations and cardinality, as this will be part of the operational dynamics of the model.

Each entity is characterized by its entity type. In this model, the following entity types are used:

- Agent: Identifying an active or passive agent which has agency
- Table: stored data that describes the context and predefined behavior of the agent (like the Knowledge Base). Tables could also have a temporal property where data of each runtime instance is stored in a workspace, that will be initialized in the next run.
- Folder: Group of tables

For the managed process, the main agents will be described and are unique for that system. The Generic MAPE-K ELV CRP model will contain standardized entities.

Agents

- “Agent in managed process”: the core agent that acts in the managed system. So, this is the object to be managed.

Agent in the Beer Game

Subject: The agent that has an active role in the supply chain. At least one agent per agent type (echelon) needs to be active, but multiple agents of the same agent type can be active.

- MAPE-K: Groups the Monitoring, Analysis, Planning, Execution, Learning, Evaluation, Validation and CRP Exchange agents.
- Monitor: Agent which will take care of scanning the context and the behavior of the managed process and translates this into symptoms.
- Analyze: Agent to pick up symptoms (generated by the Monitor agent) and apply policies from the Knowledge Base to generate Request for Changes (RfC).
- Plan: Agent which translates RfC-s into actions plans, using data from the Knowledge Base about best practice patterns that need to be used.
- Execute: Agent to apply the action plan in the orchestration layer. This will result in a possible change in the managed process.

Tables

- KB Env – Folder: Contains all configuration tables that represent knowledge of the operational system or Environment
 - o Agent Type: What are the different agent types and their characteristics.
 - o Agents: The initial list of agent identities that will be part of the simulation.
 - o Events: What are the possible states of the managed process.
 - o Event Type: The different Event types that can occur, to be able to translate from an agent agnostic to an agent specific event.
 - o Process Repository: All unique process patterns, including its type, start- and end events.
 - o DB Learning Technique: What Learning technique is used. This includes the agent specific learning sensitivity indicator and the activation of ML techniques (future feature).
 - o Policy Type: Definition of the policy types. Initially this is a process pattern policy, linked to the applicable process patterns that are stored in the process repository and the decision policy, that represent the business rules.
 - o Policy Value Range: Definition of the number range for new policy identifiers.
 - o Policy: List of all policies (all policy types) with its properties like the policy object type, attribute value, score and previous results. During runtime new policies can be added, activated or deactivated.
 - o Policy Score: The moving average score of each policy, based on earlier results in the operational environment.
 - o Trust Group: What are the possible trust groups that are active in the agent's environment. One, multiple or no trust groups can be assigned to the agent.
 - o TG Agent: To what Trust groups is the agent linked to?
 - o Change Log: All the changes that are applied to the knowledge base during runtime.
 - o Normalizer: All normalizers translate system specific events into agent specific events
 - o Mask: All events that are temporarily masked because of an agent specific reason.
 - o Filter: All events that are blacklisted for a specific reason.
 - o Symptom: All Symptoms that identify a specific system situation, based on the system events.

System Knowledge of the Beer Game

Examples of the Beer Game System knowledge are:

Event: Customer orders 2 crates of beer at cycle 1

Event Normalizer: Two different orders from agents in the retailer echelon to the wholesaler can be normalized as a retail order instead of an agent specific order

Policy: A set of sequential process patterns are selected to identify a process flow. Or the related business rules for these process patterns are selected by decision policies, linked to the higher-level process policy type.

Process Repository: The process patterns for Ordering beer, Calculation of stock levels or delivery of orders to customers are listed.

- KBMon – Folder: Contains all the tables that create the Knowledge Base for Monitoring. The table content is unique for each simulation run. Key attributes identify agent specific records.
 - o Event Collector: Select what Events are relevant for the agent to be collected
 - o Event Normalizer: Translate the Events from the operational environments to the events that are known by the agent.
 - o Event Filter: Filter those agent specific events that are applicable for this control cycle. Temporarily, events can be filtered out for example by blacklisting
 - o Event Mask: Mask those events that are relevant, but are left out of analysis for a specific purpose
 - o Event Aggregator: This Frameself element is not relevant for this model
 - o Event Symptom: Translate the Event to a generic agent specific symptom.
- KBAna: Folder that contains all the Knowledge Base tables to support analytics.
 - o Symptom Policy: Translate agent specific symptoms into agent specific policies that are applicable. This can be process pattern and decision policies.
- KBPla: Folder that contains all the Knowledge Base tables to support planning activities.
 - o Used from the Environment knowledge base
- KBExe: Folder that contains all the Knowledge Base tables about the deployment and orchestration of the executable process.
 - o Used from the Environment knowledge base

State Variables

The ABM model will have several states. This will be on both model and agent level.

- Model:
 - o Initial: Model is not running and waits to commence.
 - o Running: Model is running, and agents are acting.
 - o Paused: Model was running and is paused externally.
 - o Stopped: Model was running and finally stopped.
- Agent: This is the core agent of the main model which in this case runs the managed process.
 - o Initiated (source): Agent is created.
 - o Wait: The agent is created and waits to commence or resume the process.
 - o Running: The agent will act in the process.

- Removed (Sink): Agent is removed from the model.
- Context: This are the process specific states that will be qualified as events in its environment

Beer Game specific agent states

- *Wait for the simulation: Rider has been created and waits to start the ordering process.*
- *Active in order processing: Executing the MAPE-K control cycle to define the quantity to delivery and the purchase order content.*
- *Active in Learning: Executing the ELV cycle to learn new policies or activate previous knowledge*
- *Active in Knowledge Exchange: Executing the CRP cycle to share knowledge with other agents.*
- *Paused: As only one agent can run the MAPE-K ELV CRP cycle at a time, the other agents will pause their work. Reason for this is that the original beer game only runs in sequential (or only parallel for the agents in the same cycle).*
- *Stopped: Agent is ready to run the full simulation*

Scales

The scales define the space dimensions for the model. This can be both spatial, logical, temporal and hierarchical spaces.

Spatial space defines the geographical room where the agent can behave, restricted by physical constraints. Logical space defines the degrees of freedom for an agent within the model according to the rules of engagement and model rules, principles, and morality. Temporal spaces define the timeframe in which the agent acts. For example, the duration of the action, the length of the simulation (number of runs) or time granularity (seconds, minutes, hours, days, etc.). Hierarchical scale refers to the context in which the agent is active. When the agent acts alone or in a group it will be characterized as a hierarchical scale. This scale type is a unique type and origins from the principles of Complex Responsive Processes.

In a process, the process model defines the scales in which the agent will act (logical). When physical element is used (movement between places, use of modes of transport and storage) the physical space will become relevant. The temporal space in this context relates to the duration of the process. Acting in groups, in this model qualified as trust groups, or alone indicates the hierarchical scale characteristic.

Scales of the Beer Game

Agent _>Temporal: The original Beer Game scales are applied to be able to compare the results. For this reason, a cycle of 1 week will be applied in which agents can take actions. Interaction between agents will be decoupled as information will be delayed with at least one cycle.

- *Trust Group -> Hierarchical + temporal scale*
Knowledge Requests and exchange of policies will be applied within each trust group. However, as the scales are temporal, it is defined as a hybrid model: agents can push requests (gestures) and knowledge (response) to the other agent's workspace.

Process overview and scheduling

The managed process and the MAPE-K control loop consist of several steps. The definition of the managed system is instance specific, while the control loop is defined generically. However, during the execution of the process, the Knowledge Base will be used, which contains a fixed metadata structure to facilitate the decision-making process and is configured for its specific context. Configuration will be done by the simulation executor during initialization of the model. During initialization, the several metadata tables are initialized with characteristics of the managed process, like business logic, state phases or environmental entities. Also, the initial state of the Knowledge Base is defined as the identification of actual events, normalization rules, translation to symptom or execution plans. This knowledge is applied to control the process. In addition, when the MAPE-K learning processes are running, this Knowledge Base will be updated, where the results of the managed process are used as validation data.

Sub model Managed Process

The managed system could be any process which has to be controlled. This could be a physical process where the executing agent will act in a physical context. On the other hand, it could be a virtual process like running financial administration or planning a supply chain. In all those processes the system will be controlled by the MAPE-K ELV CRP model and information from the environment is gathered and processed to support the appropriate action to take. This action is deployed in the managed process and hence will change the context in which the managed process behaves. As other agents run the same processes in parallel, this will result in a dynamic system with a continuously changing environment which could be defined as a Complex Adaptive System.

The sub model of the managed system could be modeled with different techniques. A proven technique is the Business Process Modeling Notation (BPMN) and a Process Execution system (BPMS). With this technique, processes are modelled in a de facto standard with BPMN symbols and explicit modeling rules. The managed process could have a straightforward one-level structure where events always result in fixed results. In this case, the environmental dynamics are applied by selecting the appropriate process. In other situations, the model will contain one or more gateways which represent variability in the process. In this situation, processes can have different operational variation and are driven by the agents' decisions or fixed business logic (Split). Or, in this case, two different process flows are merged before it is continued as one instance (Join). Also, the managed process could be decomposed of several sub processes. During execution a decision can be taken to activate a single instance of the sub process. In this case business logic or agent decisions will drive the process composition [28].

When the process is modelled in BPMN, it can be deployed to the execution system (BPMS). During deployment the BPMN model is compiled into an executable process, based on an XPDL format within a BPMS platform [29]. The execution layer will connect to the user interface, runs the embedded logic or calls interfaces (API-s or interface messages) to other applications.

The managed process is the baseline for each MAPE-K ELV CRP cycle. This process is context specific and will contain different BPMN symbols to model the operational sequence of activities. Decisions are defined as gateways; activities are assigned to roles and process decomposition is done by the definition of sub models. Interaction with other processes is symbolized by start, intermediate or end events. These process models could be imported from existing BPMN definitions and extended by the MAPE-K ELV CRP processes.

Also, as the model is based on BPMN, the processes could be embedded in a BPMS orchestration engine, which is comparable with a smart middleware execution platform. API-s to external applications, scripts and user interface will make the system executable. However, in simulation mode, this execution could be virtualized without operational impact.

A generic model of a managed system with a MAPE-K ELV CRP control loop will contain the events, activities and gateways of the managed process, a link to MAPE-K. MAPE-K is called upon, including the CRP exchange process.

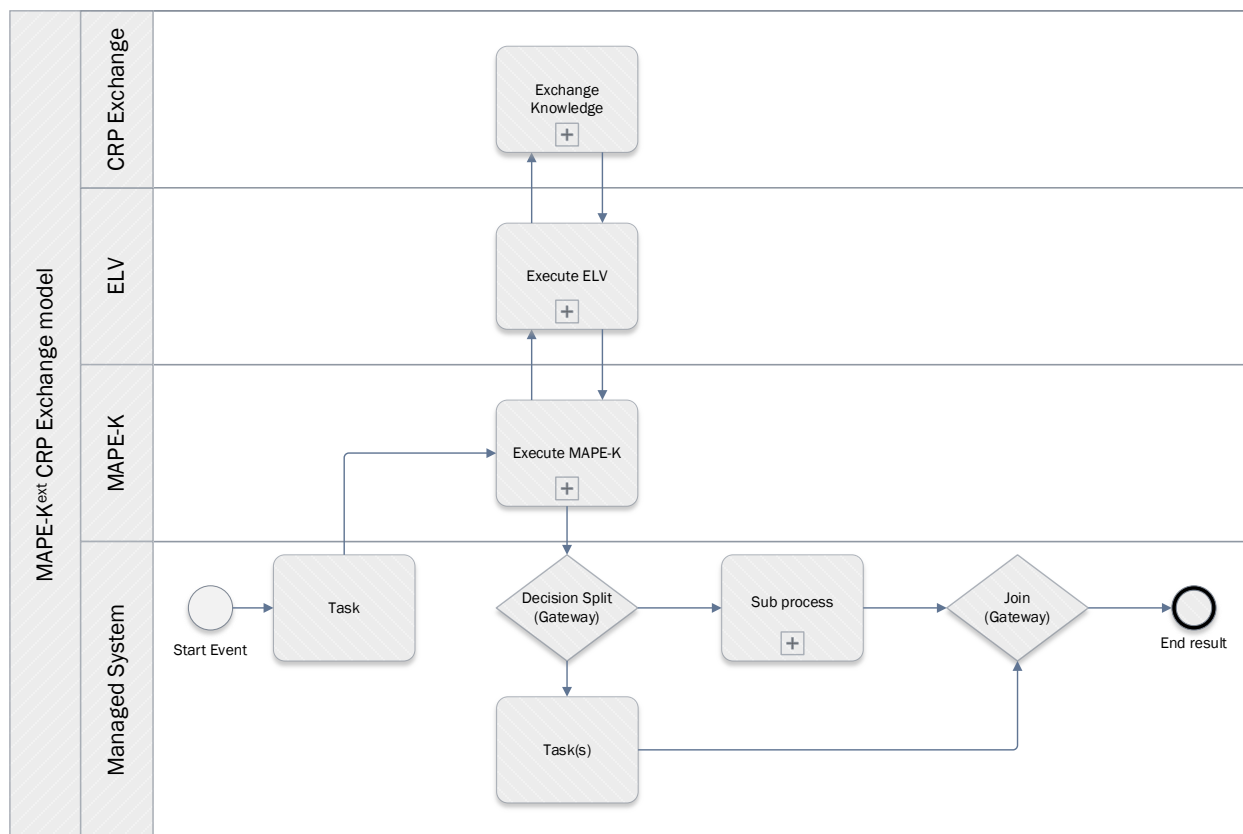


Figure 2: Generic MAPE-K ELV CRP model in a managed process

The Beer Game Model

This model defines a four-echelon structured supply chain and includes business logic and facilitates tactical decisions by the agent. To drive these decisions, the MAPE-K control loop is modelled as a sub-process.

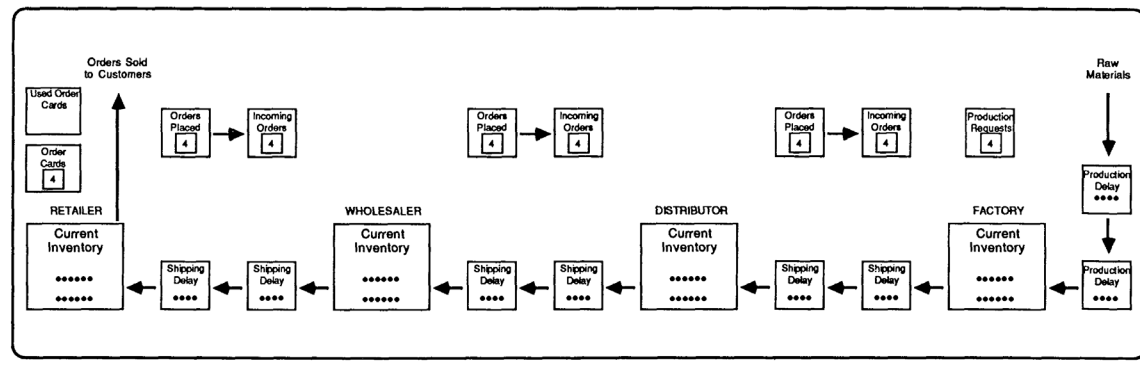


Figure 3: Beer Game Board (as presented in Sterman) (Sterman, 1989)

Orders will flow from left to right, while the physical flow will be from right to left. In each cycle (week) an agent can take action by receiving customer orders, receive deliveries, deliver shipments, calculate stock, create purchase orders and produce new products (only the manufacturing agent).

This process can be modelled as follows:

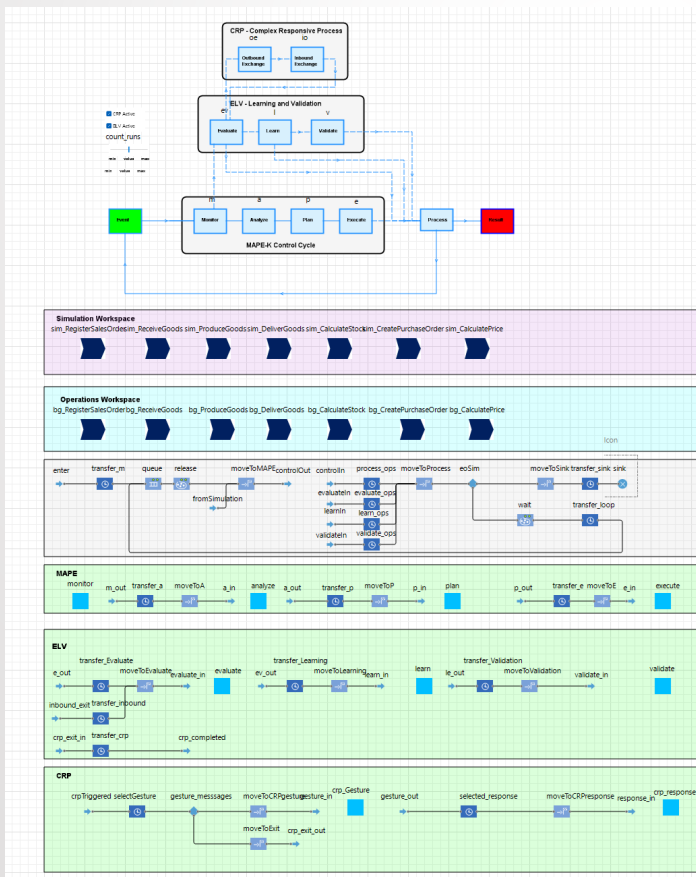


Figure 4 Make model with MAPE-K ELV CRP

Sub model MAPE – K loop

In the MAPE-K sub model the agent's decisions are defined that will drive the managed process. During this process, the environment is scanned, interpreted, analyzed, and translated into actions to change. As this control loop is agent- and process instance specific, it can act immediately on changed conditions in the environment and applies the logic stored in the Knowledge Base, in favor of the agents' objectives.

The process model is generic for all managed processes and will use context specific data which is retrieved from the management process and stored a generic data model. These data are combined with the configuration data for the model and the initial or updated Knowledge Base to support monitoring, analysis, planning and execution.

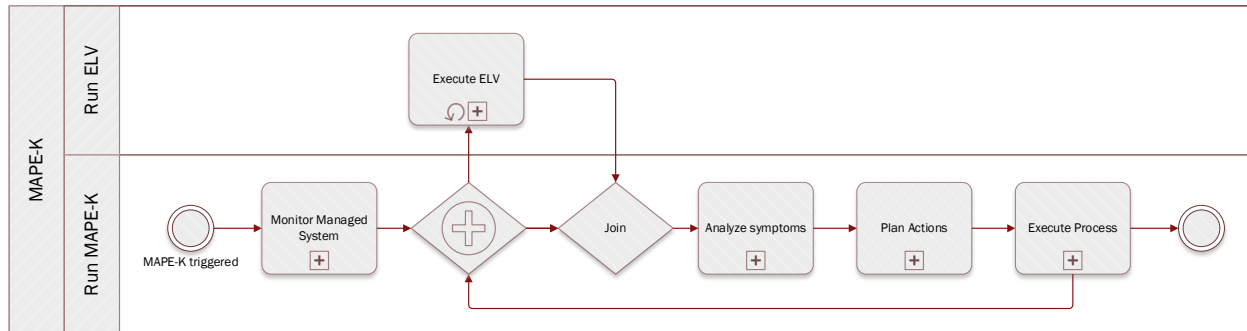


Figure 5 MAPE-K control loop

The process contains sub processes that define a fixed set of activities to execute the tasks.

The process is triggered by a MAPE-K intermediate event, raised from the managed process. Then, the series of sub processes is executed: monitor the managed system and translate into symptoms, analyze the symptoms, and define request for changes, plan the corresponding actions and execute the updated process. For each run the ELV process is executed to Learn, Evaluate and Validate the changes in the Knowledge Base. After this process, the instance will return to the managed process for execution.

The MAPE-K process can be identified as the *exploitation* of the agent's knowledge, retrieved from the Knowledge Base.

Sub model ELV learning capability

For each MAPE-K cycle the learning capability is triggered. This will result in an updated Knowledge Base where lessons learned from earlier actions are stored. This process takes care of *exploration* properties of the system and drives stochastic behavior and results in emergence of the total system. Agents are using the updated knowledge to make decisions in the MAPE-K cycle. Also, a link to the CRP Exchange capability is made to facilitate collaborative learning.

The first step is to learn about the actions taken earlier and their results in the managed process. Did the action result in the expected outcome, or did it fail? With these results, the Knowledge Base policies will be marked as liable for change. With these lessons learned the possible alternative policies are considered. The most appropriate policy will then be selected. In this step, machine learning capabilities can be used to improve the library of alternatives. In the last step, the updated policy will be evaluated, and the expected possible outcome is defined. With this result, the Knowledge Base policy is updated. When no good alternatives are available, no further action takes place. After completion of this process, the ELV process is completed.

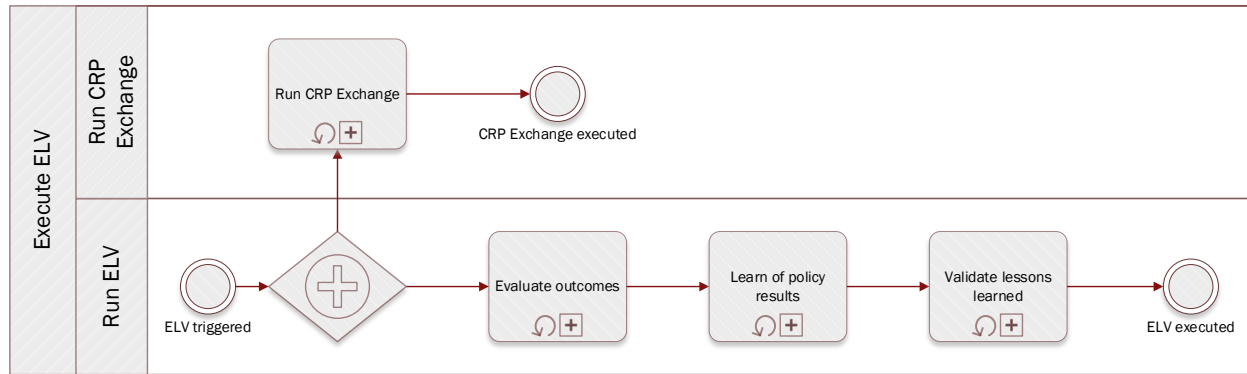


Figure 6 ELV process with CRP embedding.

Sub model CRP extension

Proven MAPE-K models in an ABM have an agent focused locus. As this is an intrinsic value of an agent-based system it is common use to keep the agency and learning capabilities within the reach of information that is available for this individual agent.

What we learn from human systems, the exchange of ideas, concepts, techniques and experience is a strong capability of social behavior. When a human learns, it will use external information and embed this in its personal mental model and skill repository. Inter-agent exchange will propel the development of knowledge and contextual understanding. So, when we implement the inter-agent information exchange in management systems with virtual agents, we could speed up the learning capabilities and improve decision making in a continuously changing environment. This will support the explorative character of the system and drives complex, stochastic and emergent behavior.

During the learning process in MAPE-K the *CRP Exchange* is triggered. This CRP Exchange is bi-directional: share knowledge with other agents – *Gesture*; and receive knowledge from other agents – *Response*. This knowledge is retrieved from and stored in the agent's Knowledge Base.

To exchange data with others several elements are critical. First, a selection of knowledge message types must be selected. Only those records in the Knowledge Base which have been changed will be selected.

Hence, the group of agents must be selected to whom the messages will be shared. This will be done in the Gesture CRP Exchange process and is shared within one or more specific trust groups. As the specific policy id-s are not harmonized, the sending agent will share the object type and attribute of the policy, which is harmonized in the System Knowledge Base of each agent. The sending agent will also share its best performing policy.

When the agent has shared its knowledge with the trustee agents, the next step is to receive external knowledge from the trust group. In this case, also the selected message types to retrieve are defined, based on the message types (object type + attribute) that are of specific interest of the agent, defined in the Knowledge base records. For all relevant message types, the Response CRP Exchange is processed. In this process, the agents in the Trustee group are asked for knowledge on the message types. After a gesture and response cycle, the external knowledge from the trustee group is updated in the agent's knowledge base.

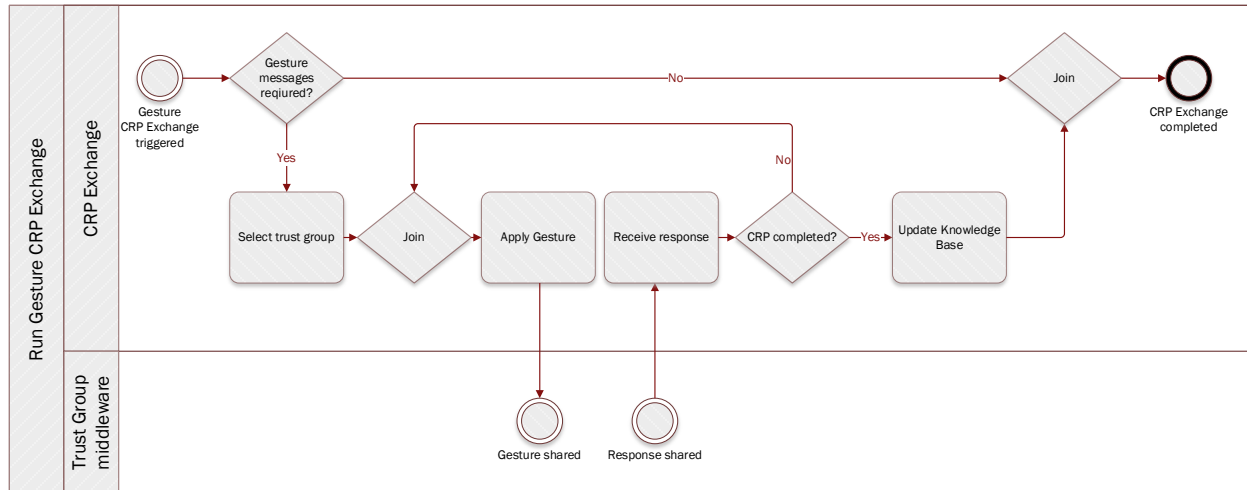


Figure 7 CRP Gesture Exchange

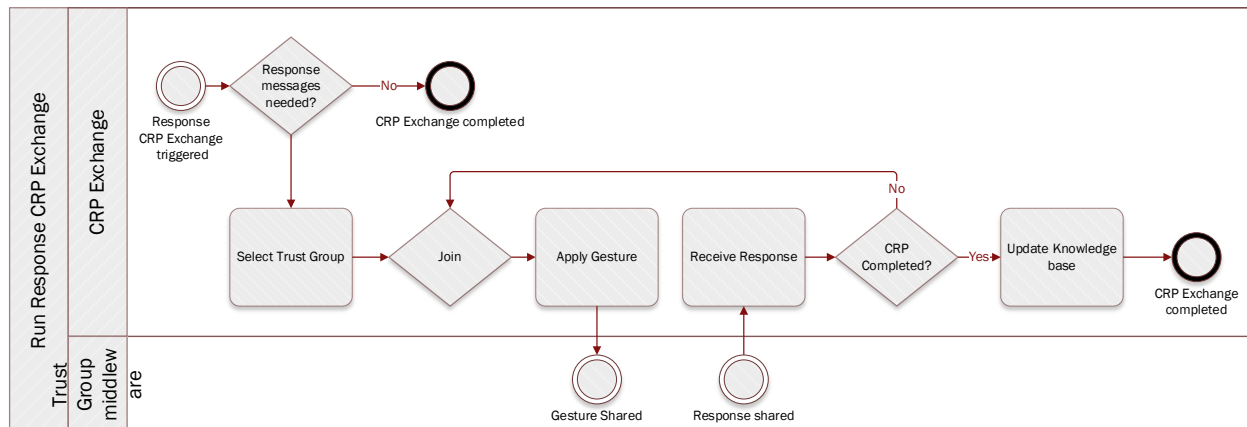


Figure 9 CRP Response Exchange

After this CRP Exchange process, the Knowledge Base is updated and ready to use in the next MAPE-K and ELV cycle.

Design Concepts

Basic Principles

The MAPE-K ELV CRP model is based on clear relations between the components of the managed system, the BPM and the simulation including the control loop.

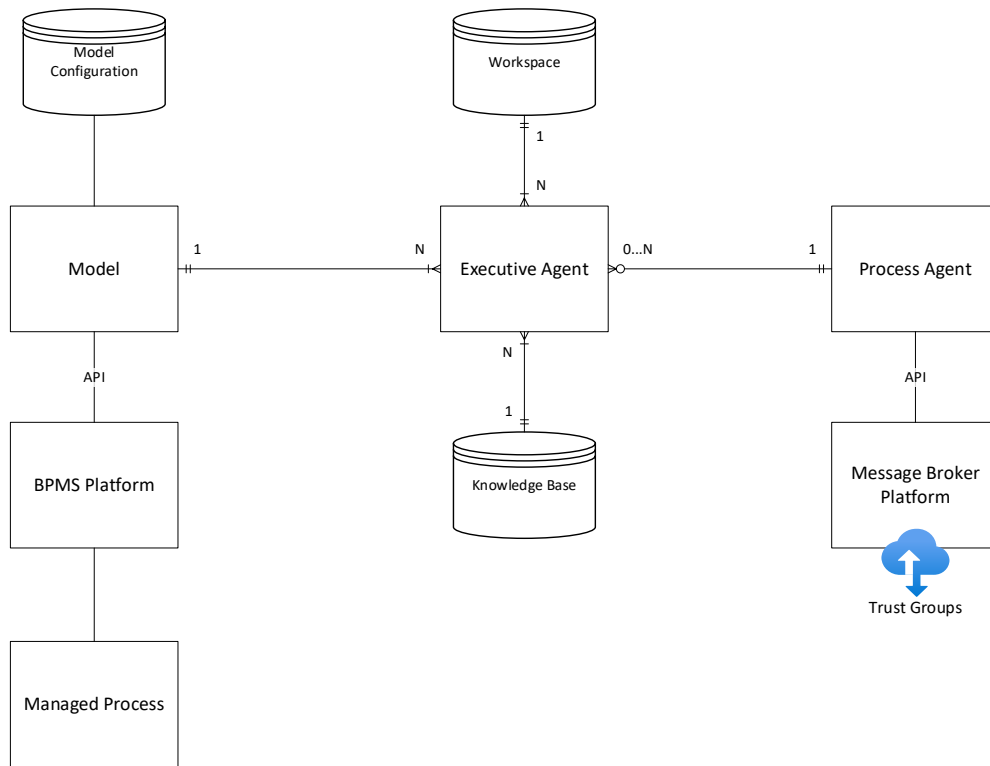


Figure 8 Model architecture

Managed Process

The operational environment where the processes are deployed, and events occur. These events represent unique characteristics of the context in which the process agents act and are the trigger for the monitoring processes in MAPE-K.

BPMS platform

The Business Process Management System (BPMS) is a process engine to execute process flows that are defined with the BPMN convention. This convention consists of a clear taxonomy where each symbol represents a process activity or state. As the process flows are visualized, this BPMN convention enables low-code development. When processes are deployed to the process engine, they are compiled into a XPDL orchestration script. This is an executable script on the runtime environment.

For this MAPE-K ELV CRP model it is critical to apply a loosely coupled process architecture with atomic process patterns. These patterns are designed to achieve a specific, single objective. Also, due to the use of intermediate events, they can be linked to each other in a flexible and dynamic manner. In this way, the loosely coupled principle is respected, and processes can be assembled for each unique process instance. This assembly will be a one-off and is designed during the MAPE-K control cycle.

Model

The simulation model can be seen as the digital twin of the managed process. This model contains all possible process patterns and its relationship with other main process patterns. For this research, the model is agent-based and could contain sub-processes. These sub-processes represent process agent to achieve a specific task, in this case: Monitoring, Analysis, Planning and Execution in MAPE-K, Evaluation, Learning and Validation in the ELV and Outbound and Inbound exchange in CRP.

The model is linked to the BPMS bi-directionally via API-s: process states (events) are retrieved from the managed process and execution plans are deployed to the process orchestration engine. Deployment takes place by pushing the process pattern script to the BPMS. With this, the process flow visualization can be generated, and the process XPD, XML based file [30] is compiled. With this file, the process can be executed.

Model Configuration

To represent the managed process in the digital model all critical characteristics should be configured. This configuration is done by entering the context states in static tables. Also, the process objectives, number of agents and other relevant model characteristics are stored.

Knowledge Base

As the MAPE-K cycle needs to have a kick start, the initialization of the Knowledge Base tables in the KBMon, KBAna, KBPla and KBExe domain is done, where events, symptoms, symptom policies, RfC policies, and other KB details are entered. These data are the starting point for the first runs of the process.

Workspace

During the MAPE-K, ELV and CRP control processes phased monitoring, analyzing, planning and execution activities are processed. Also, the learning and inter-agent data exchange is executed. During these processes, the intermediate data is stored in workspaces and during can be updated in each process step. Finally, the workspace results can be stored in the Knowledge Base. Also, for each new process, the workspaces are initialized.

Executive Agent

Each process will be executed by the *Executive Agent*. This is the actor to run the process and runs the managed process and the MAPE-K ELV CRP control cycle. Each model consists of at least one agent but could contain multiple agents. That implies that the cardinality between the model and the executive agent is one to many.

Process Agent

The managed process will have one or many sub processes, as part of the process decomposition. By decomposing a process, it is possible to activate atomic process patterns. This is required to enable a loosely coupled architecture and runtime process composition. As a result of this, each process becomes instance specific.

Also, the MAPE-K, ELV and CRP process is defined as a multilayer process decomposition. Each sub process defines the process pattern and is triggered by the higher-level process in the MAPE-K cycle. In the managed process, the process patterns could also be modeled as sub processes (which are unique process agents). These sub processes are triggered by intermediate events that are selected as an outcome of the MAPE-K cycle.

Message Broker Platform

To enable the inter-agent data exchange during the CRP process, a blackboard communication technique is used, where the agent is subscribed to message types. These data are exchanged in XPD format and consists of request and receive functionality. In CRP terminology this is called the *Gesture* and *Response* interaction. The message broker platform is used by all other agents which will communicate with each other. To canalize the communication, the agents are grouped in *Trust Groups*. These Trust Groups are maintained in the Knowledge Base of the agent, where the preference for each group is stored. When

requests are sent to the message broker platform, this will be published to one or multiple Trust Groups. When data is received, this will be retrieved from specific Trust Groups.

Emergence

This MAPE-K ELV CRP model will define applicable policies or relevant process patterns. For that purpose, the ontology is pattern-oriented [31], where patterns of processes are linked to create instance specific process flows. Policies will drive gateway decisions. Both the process instantiation and policy algorithms drive the emergent nature of the system. As the CRP component results in intensive agent-collaboration, influences from the context space will directly drive the emergent behavior of the agent specific process.

Adaptation

Adaptation in this model will take effect on two levels: the operational control cycle (MAPE-K) and the learning cycle (ELV), supported by CRP as a knowledge accelerator.

In operational control, decisions in the operational environment are prepared and analyzed. This results in the instance specific, real-time adaptation of business rules and application of process patterns. To facilitate this, process orchestration and BPMS engine are required. Also, the composition of processes should be atomic and fine grained, in a loosely coupled architecture. This enables maximum flexibility and unique process construction.

The other level of adaptation is the learning cycle, where RfC policies and process patterns are evaluated, reconsidered, removed or added. During these learning activities, AI techniques could be added. Also, the knowledge of other agents can be exchanged via the CRP Exchange, where knowledge of RfC policies and process patterns is shared amongst Trust Groups. This will result in adaption of the Knowledge Base, which will have direct impact on the operational process control cycle.

Objectives

The objectives of the model can be split into process objectives and policy/process pattern expectations. As the objective of the process is the foundation for success, the policy expectation should contribute to the process objectives.

During the modeling of the process, the process indicators are defined. These indicators are part of the strategic KPI structure of the organization. These KPI-s are converted into Process Indicator, which defines the success of the process. The process indicator is SMART (Single, Measurable, Attainable, Relevant and Time-Bound) [32] and so can be easily compared with the actual outcomes, in this model called “events”.

The policy and process pattern expectation identifies the success of each adaptation. This expectation is used to evaluate the selected policies and process patterns during the MAPE-K control cycle. When the actual result is better than the expectation, it is qualified as a positive contribution. In case the result is worse than expected, the qualification is negative, while equal is qualified as neutral.

Learning

The ELV process covers the evaluation of the actions taken, the learning of new variations in policies and process patterns and the validation of this new knowledge. To support learning, several AI techniques could be used from unsupervised to supervised and reinforcement learning. For all new knowledge first, a validation is done to be sure that the new knowledge is contributing to an improvement of the results. This is done by a higher-level simulation (simulation of the simulation). Only when the learned knowledge contributes positively to the (expected) process result, it will be active.

Also, learning will be accelerated by the exchange of knowledge from other agents (CRP Exchange).

Prediction

For each policy and process pattern, the expected result is defined. Based on this a simulated prediction and an expected actual result is calculated. For each new control loop, this expectation is calibrated with the actual results. In addition, during higher level simulation a prediction of the expectation is created to have a qualified starting point for new knowledge.

Sensing

The MAPE-K control loop starts with a monitoring task. This task will retrieve all relevant events in the managed process and normalize this to be able to analyze the behavior of the process. Sensing the managed process not only relies on the process events, but also the topological characteristics of the environment. On a change of configuration, the learning cycle is activated.

Interaction

This model is agent-based and includes an intrinsic process for knowledge sharing between agents. This is called the Complex Responsive Process Exchange (CRP), based on principles of Hegels dialectics ([33] 33). The source of the CRP is found the Social Complexity Science in the work Stacey [27]. The key principle of CRP is that agents interact and share information to create their own, instant reality. As a result of these dialectics, information is created, and decisions are taken.

Stochasticity

The MAPE-K ELV CRP model is based on a complex social network where emergence takes place because of agent interaction. During the evaluation process, a stochastic element is built in, where triggers for learning are not only driven by changed configuration but also due to random learning triggers. As this randomness will take place in each agent's evaluation process and agents interact, this randomness could result in exponential growth or decrease. In the model definition, this randomness can be altered to manage the entropic characteristics of the network.

Collectives

Each agent is acting on its own dataset, retrieved from the managed process environment and its knowledge base. During runtime, all relevant data is collected in workspaces. These workspaces are internal tables and store the data that is relevant during the process. During its lifetime, the data can be enriched, maintained, or removed, because of process activity. For each new process control cycle, the workspaces are initialized.

Observation

The concept of the digital twin implies that during execution of the process the simulation is running. From this perspective the managed process by itself will be the observing agent who controls the simulation. The need for simulation is triggered by the process instance while the results are embedded in the process itself. However, this system entails the self-organizing characteristics where now authority is assigned: the observer becomes the observed one, so the model will observe the management process as well ([34] p.13).

Details

Initialization

On each run of the model, all workspaces and parameters will be initialized. This applies to the policy and the symptom policy table. The reason for this is that it will cancel earlier simulation results and initialize

the start values. On the contrary, the Knowledge base tables will not be hit so the agents earlier experience will be secured and ready for later use.

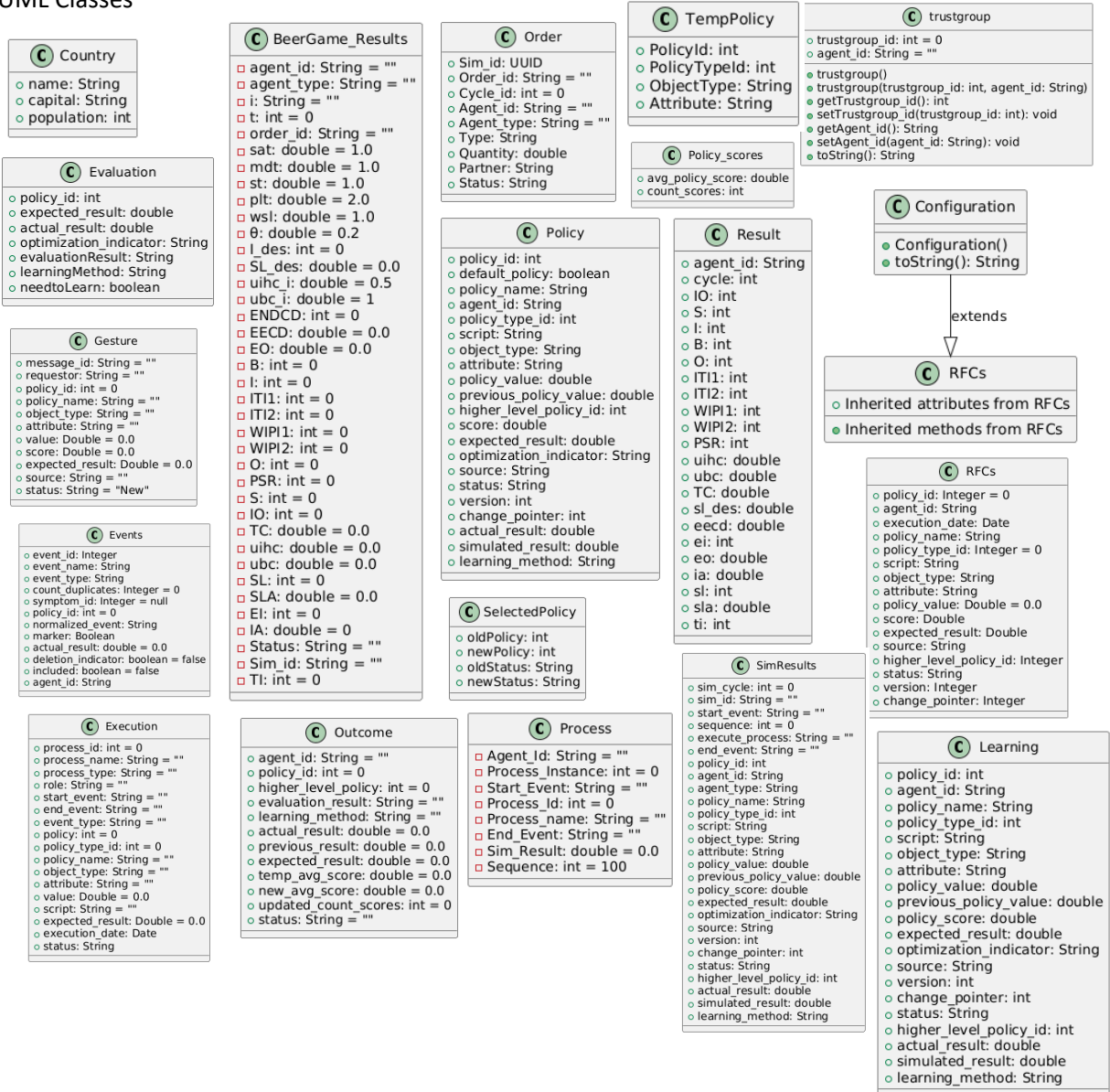
Input Data

When the model is configured all model characteristics are defined in the Knowledge Base Environment tables. These configuration tables are stored in the Model domain of the database. These data are the foundation for the execution of the digital twin.

Triggered by the managed process, the simulation starts, and the relevant data are retrieved from the managed system. This includes the event data, process configuration and will be enriched by external data from other agents in the CRP exchange.

UML Class Diagrams

UML Classes



Monitor

Event Filter

```
+ agent.eventWorkspace(Event)
+ event_filter(t)
+ fGetEventFilter(agent)
+ fDiscardEvent(agent)
```

Event Masker

```
+ agent.eventWorkspace(Event)
+ idNaam
+ event_mask(t)
+ fGetEventMask(agent)
+ fIgnoreEvent(agent)
```

Event Normalizer

```
+ agent.eventWorkspace(Event)
+ fGetEventNormalizer(agent)
+ fTransformEvent(agent)
```

Symptom Sender

N/A

Event Aggregator

```
+ agent.eventWorkspace(Event)
+ fMergeDuplicates(agent)
```

Event Collector

```
+ mainActualEvents(Event)
+ process_repository(t)
+ agent.eventWorkspace(Event)
+ event_normalizer(t)
+ fGetEventCollector(agent)
+ fCollectEvent(agent)
```

Monitor Manager

```
+ agent(Subject)
+ agent.eventWorkspace(Event)
+ agent.rfcWorkspace(RFCs)
+ agent.planWorkspace(Plans)
+ agent.actualEvents_New(Event)
+ mapEventTriggered(Enter)
+ EventCollector(Delay)
+ EventFilter(Delay)
+ EventMasker(Delay)
+ EventNormalizer(Delay)
+ EventAggregator(Delay)
+ SymptomGenerator(Delay)
+ SymptomRetrieved(SelectOutput)
+ noSymptomSent(Bit)
+ SendSymptom(Delay)
+ SymptomSent(Bit)
+ fOrchestrateAnalysis(agent)
```

Symptom Generator

```
+ agent.eventWorkspace(Event)
+ event_symptom(t)
+ ffinderSymptom(agent)
```

Knowledge Base Communicator

N/A

Analyze

Policy Resolution

```
+ rfcWorkflowSpace(RFCs)
+ policy(t)
+ fGetPolicyOnFilter(agent)
+ fGetPolicyOnFilter(agent)
```

Policy Validator

```
+ agent.eventWorkspace(Event)
+ symptom(t)
+ symptom_policy(t)
+ event_policy(Event)
+ policy(t)
+ agent.rfcWorkflowSpace(RFCs)
+ fValidatePolicy(agent)
+ fRetrievePolicy(agent)
```

Symptom Receiver

N/A

Knowledge Base Communicator

N/A

Analyzer Manager

```
+ agent(Subject)
+ SymptomManager(Enter)
+ SymptomReceiver(Delay)
+ PolicyValidator(Delay)
+ ActiveSymptom(SelectOutput)
+ NoSymptom(Bit)
+ PolicyRetrieved(Delay)
+ RfcSelected(SelectOutput)
+ NoRfc(Bit)
+ RfcGenerated(Delay)
+ SendRfc(Delay)
+ RfcSent(Bit)
+ fOrchestratePlan(agent)
```

RFC Generator

```
+ agent.rfcWorkflowSpace(RFCs)
+ fGenerateRfc(agent)
```

RFC Sender

N/A

Plan

Policy Transformer

N/A

RFC Receiver

```
+ agent.rfcWorkflowSpace(RFCs)
+ fReceiveRfc(agent)
```

Knowledge Base Communicator

N/A

Policy Interpreter

```
+ agent.rfcWorkflowSpace(RFCs)
+ policy(t)
+ fApplyPolicy(agent)
```

Plan Generator

```
+ agent.rfcWorkflowSpace(RFCs)
+ (Applied) Policy?
+ ActionPlan
+ RFC
+ fGenerateActionPlan(agent)
```

Plan Sender

N/A

Planner Manager

```
+ agent(Subject)
+ RfcSent(Enter)
+ RfcReceiver(Delay)
+ PolicyInterpreter(Delay)
+ PolicyTransformer(Delay)
+ PlanGenerator(Delay)
+ PlanSender(Delay)
+ checkELV(SelectOutput)
+ plan_ELIV(Exit)
+ PlanSent(Bit)
```

Execute

Knowledge Base Communicator

N/A

Workflow Engine

N/A

Action Formatter

N/A

Action Dispatcher

N/A

Plan Receiver

N/A

Action Scheduler

```
+ agent.executionWorkspace(Execution)
+ process_repository(t)
+ agent.planWorkspace(Plans)
+ fScheduleAction(agent)
```

Action Interpreter

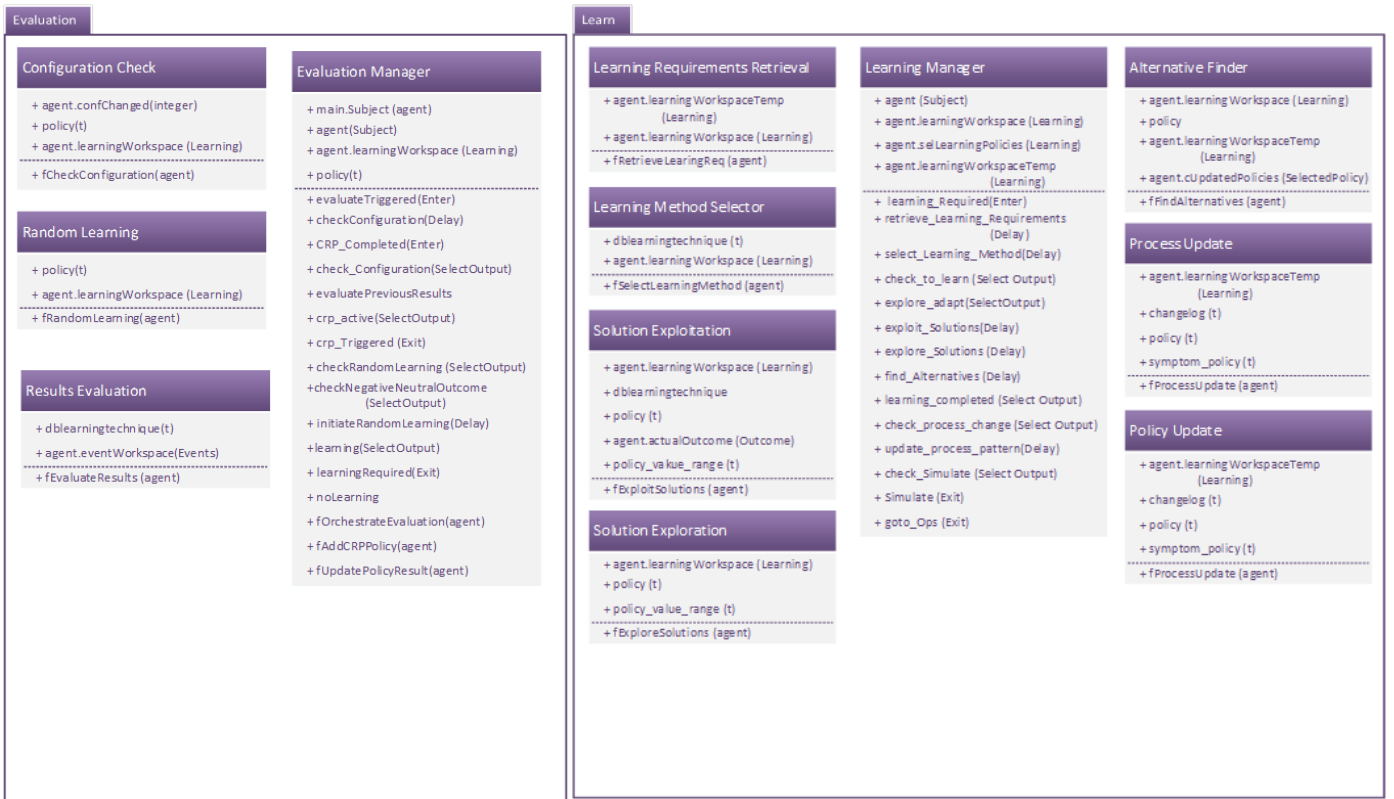
N/A

Process Orchestrator

N/A

Executor Manager

```
+ Subject(agent)
+ agent.executionWorkspace(Execution)
+ PlanSent(Enter)
+ PlanReceiver(Delay)
+ ActionInterpreter(Delay)
+ ActionScheduler(Delay)
+ NoExecutionRequired(Exit)
+ queue(queue)
+ next_Execution(Enter)
+ triggerExecution(SelectOutput)
+ ActionDispatcher(Delay)
+ ProcessOrchestrator(Delay)
+ ExecutionTriggered(Exit)
+ leave_Execution
+ fOrchestrateProcess(agent)
+ fDispatchAction
```



Validate

Process Simulator

- + agent.simulationWorkspaceInit (SimResults)
- + process_repository (t)
- + fSimulateProcess (agent)

Simulation Monitor

- + agent.simulationWorkspaceInit (SimResults)
- + agents (t)
- + policy (t)
- + process_repository (t)
- + agent.simulationWorkspace (SimResults)
- + fMonitorSimulation (agent)

Process Executor

- N/A

Results Analyzer

- + agents (t)
- + main.xxxSim (SimResults)
- + agent.simulationWorkspace (SimResults)
- + fCalculateTotalResult (agent)
- + fAnalyzeResults (agent)

Validation Manager

- + agent (Subject)
- + agents (t)
- + agent.simulationWorkspace (SimResults)
- + policy (t)
- + Simulation_Finished_In (Enter)
- + KB_Updated (Enter)
- + Runs_Finished (Select Output)
- + simulate_Process (Delay)
- + monitor_Simulation (Delay)
- + execute_Process (Delay)
- + Next_Process (Enter)
- + Process_Finished (Select Output)
- + Simulation_Finished_Out (Exit)
- + toSimulate (Exit)
- + analyze_Results (Delay)
- + check_Outcome_Improved (Select Output)
- + check_Process_Pattern (Select Output)
- + update_Policy_Expectation (Delay)
- + deactivate_Process_Pattern (Delay)
- + deactivate_Policy (Delay)
- + check_No_More_Results (Select Output)
- + learning_Validation_Completed (Exit)
- + fResetSimParameters (agent)
- + fFillSimulationWorkspaceAll (agent)
- + fSetAgentSimResults (agent)
- + fGetAgentSimResults (Agent)
- + fExecuteProcess (agent)

Policy Updater

- + agent.simulationWorkspace (SimResults)
- + policy (t)
- + main.subject (agent)
- + trustgroups (t)
- + policy_score (t)
- + changelog (t)
- + fUpdatePolicy

Process Pattern Deactivator

- + agent.simulationWorkspace (SimResults)
- + policy (t)
- + symptom_policy (t)
- + main.subject (agent)
- + trustgroups (t)
- + changelog (t)
- + fDeactivateProcess (agent)

Policy Deactivator

- + agent.simulationWorkspace (SimResults)
- + policy (t)
- + symptom_policy (t)
- + main.subject (agent)
- + trustgroups (t)
- + changelog (t)
- + fDeactivatePolicy (agent)

CRP Gesture

Trust Group Selector

- + agent.agent_TrustGroups (trustgroup)
- + tg_agent (t)
- + trustgroups (t)
- + fSelectTrustGroup (agent)

Gesture Definer

- + agent.gestureList (Gesture)
- + agent.crp_Workspace (Gesture)
- + policy (t)
- + fDefineGesture (agent)

Gesture Sender

- + agent.gestureList (Gesture)
- + agent.publish_TrustGroups (trustgroup)
- + agent.crp_Workspace (Gesture)
- + subject (agent)
- agentsInTrustGroup (trustgroup)
- + fSendGesture (agent)

Gesture Manager

- + agent (Subject)
- + agent.learningWorkspace (Learning)
- + agent.crp_Workspace (Gesture)
- + crp_triggered (Enter)
- + select_TrustGroup (Delay)
- + defineGesture (Delay)
- + sendGesture (Delay)
- + gesture (Exit)
- + fFillCRPWorkspace (agent)

CRP Response

Response Processor

- + agent.crp_Workspace (Gesture)
- + policy (t)
- + symptom_policy (t)
- + changelog (t)
- + fProcessResponse (agent)

Trust Group Selector

- + agent.agent_TrustGroups (trustgroup)
- + tg_agent (t)
- + trustgroups (t)
- + fSelectTrustGroup (agent)

Response Definer

- + agent.responseList (Gesture)
- + agent.crp_Workspace (Gesture)
- + policy (t)
- + fDefineGesture (agent)

Response Manager

- + agent (Subject)
- + gestureIn (Enter)
- + processResponse (Delay)
- + select_TrustGroup (Delay)
- + defineResponse (Delay)
- + sendResponse (Delay)
- + responseOut (Exit)

Response Sender

- + agent.responseList (Gesture)
- + agent.publish_TrustGroups (trustgroup)
- + agent.crp_Workspace (Gesture)
- + subject (agent)
- agentsInTrustGroup (trustgroup)
- + fSendResponse (agent)

Sub model Managed System

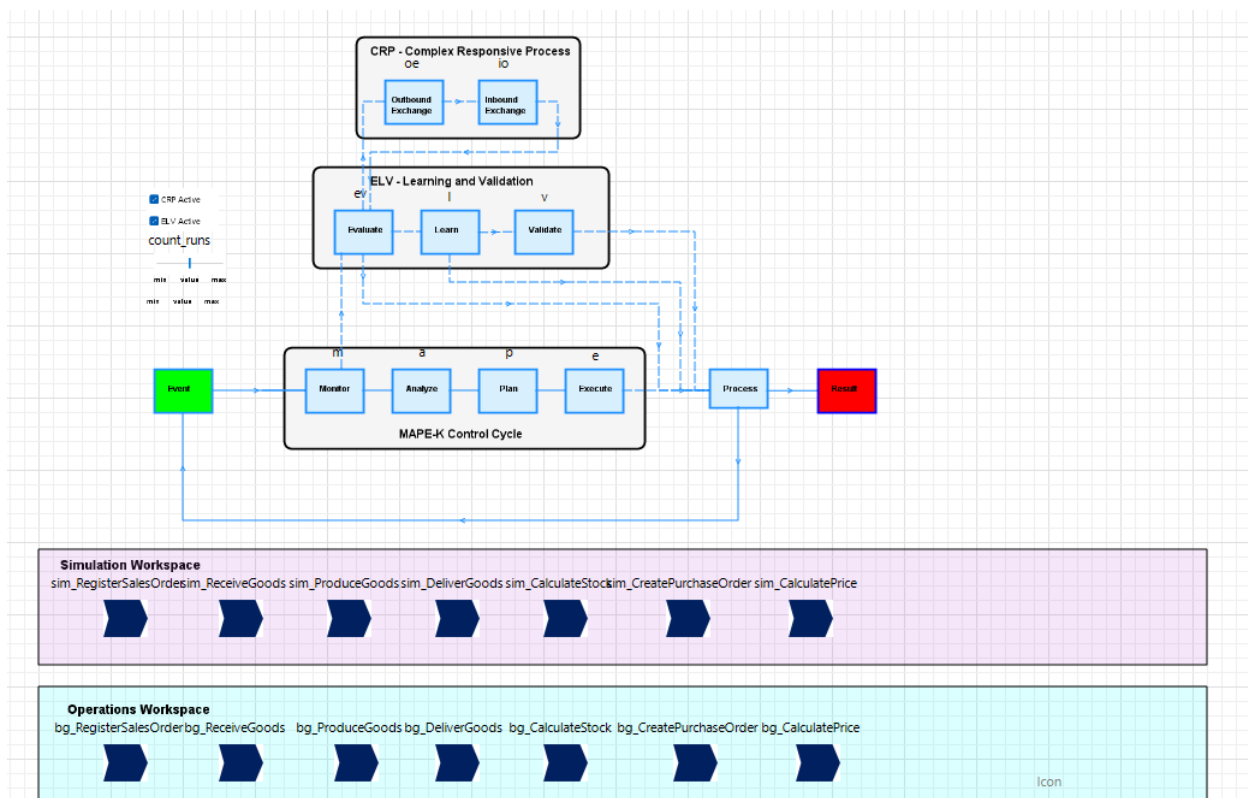
The process of the managed system has already been explained in the overview of the model. The details of this model depend on the operational context. From a generic perspective it is not possible to go into the details. However, there are a few aspects relevant:

- The process should be modeled as an agent-based model.
- The agent will act in a single instance of the process.
- The process flow will at least contain one decision (gateway to split the process).
- Multiple tasks could be grouped into sub processes. This sub process will become a new (process) agent, where the main (executing) agent will call upon.
- Start Events will start the execution of the process while End Events will stop the process instance.
- Intermediate events trigger other processes (internal or outside the organization) or can pick up external triggers to continue the process.
- When process splits come together, these will be modelled with a join symbol.
- Each symbol can be characterized by its specific nature according to the BPMN standard.

Main

The main process orchestrates the full operations of the model. As this research is based on the fictive Beer Game, these operational processes are modelled in the simulation environment as well.

The structure is defined as follows



1. **Operations Workspace:** Represents the operational process environment for the Beer Game (bg)
 - a. Register Sales Order: Record the incoming sales order from the consecutive echelon.

- b. Receive Goods: Receive the ordered goods from the previous echelon. As the Beer Game uses In Transit stock between echelons, this means receiving the Inbound Stock In Transit (ITI2/WIPI2).
 - c. Produce Goods: Only for the manufacturer who will produce the goods that are planned.
 - d. Deliver Goods: Deliver the goods that have been ordered by the customer (consecutive echelon). Goods are delivered to the outbound In Transit Stock (ITI1/WIPI1)
 - e. Calculate Stock: Calculate the quantity to be delivered to the customer, to be produced (only for manufacturer) to be bought in purchase orders or moved to between In Transit Stock
 - f. Create Purchase Order: Transform the planned quantity into a purchase order, to be ordered at the preliminary echelon.
 - g. Calculate Price: Not relevant for the Beer Game. However, this process has been modelled to show that not all processes in the repository are selected during runtime and simulation.
2. Simulation Workspace: These processes are equal to the Operations Workspace as this research is based on a fictive operational environment. The simulation processes are used to create a representation of the operational system.
3. MAPE-K Control Cycle: Trigger the MAPE processes for each cycle in the operational process and is agent-specific
4. ELV Learning Cycle: When learning is triggered (based on topology change or randomly) the full learning cycle is executed
5. CRP Learning Cycle: For each ELV cycle, the exchange of knowledge between agents is executed.

Root

In the Root, the following methods are applicable

- Method -> Orchestrate Monitor: Trigger the MAPE-K cycle for each execution run
- Method -> Retrieve Events and Execute: Fill the DB Events database from an external database while normally, they are triggered by an API from the operational environment
- Method -> Retrieve Events and Map: Retrieve the actual events from the operational environment. These are retrieved from the DB Events database and stored in the Actual Events collection, which are agent independent.
- Method -> call API: Interface to trigger the deployment of processes in the operational environment. In this research, the operational environment is fictive, so this API is not used.
- Method -> Add Stock: After running the virtual processes in the operational environment, the stock is added to the stock ledger
- Method -> Calculate Total Result: Calculate the individual costs calculated for the inventory and backorder per agent and calculate the totals.
- Method -> Fill Graphics: Retrieve all data from the operational environment and store these in the graphic tables
- Method -> Initialize: Reset the system values, internal tables and variables

The following additional collections are used to be able to run the fictional environment

- Actual Events (Events): Table of start events that are applicable for this full simulation run

- Orders (Order): Table of purchase orders that are created by the agents. As the consumer is modelled as an agent as well, all these orders will result in the incoming sales order for each echelon
- SimOrders (Order): Table of purchase orders in the simulation environment
- cBeerGameResults (Beer_Game_Results): Table of all the results that result from the fictive operational processes in the Beer Game
- cBeerGameSim (Beer_Game_Results): Table of all the results that result from the simulation runs of the Beer Game.
- Selected Agents (Subject): Table of agents that are active in the simulation run
- cOperationsOutcome (Result): Output of the Beer Game processes (orders, stock levels and costs)
- cSourceGraphics (Result): Table that stores the source data for all graphics

Sub model MAPE – K loop

The MAPE-K loop consists of the four elements Monitor, Analysis, Planning and Execution, while the Knowledge Base is used or updated.

Monitor

The monitoring process will read the characteristics of the context for each agent and translate this into known symptoms. In several steps, the events are normalized, filtered, masked and transformed. When no event is relevant for further processing, an exit end event is triggered.

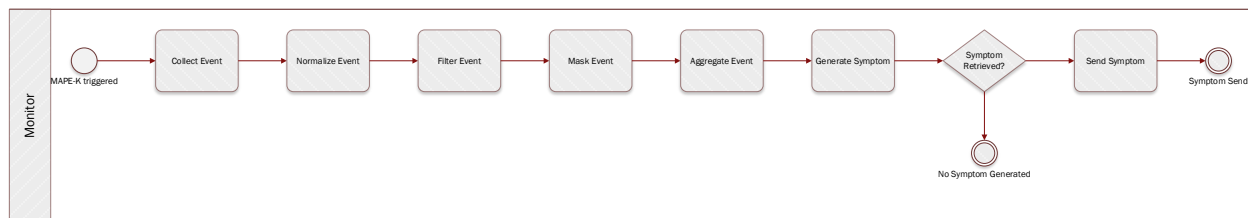


Figure 9 Monitor process

This process is orchestrated in the BPMN flow and digital twin by the orchestrateMonitor method that coordinates the execution of the monitoring process.

Activity: Event Collector

This activity retrieves the characteristics of the operational context of the management process. These characteristics are defined as *events*. As the type of events are model-specific, a function Get Event Collector is used. This will retrieve the types of events that are applicable for this run of the model. This could be none, one or multiple filters. After that, the function Collect Event is executed, where all the event types which are mentioned in the collector collection are retrieved from the operational context.

- Method -> Get Event Collector: When the model was defined the different context events that are applicable are stored in a configuration table Events. The Event types that are relevant to the specific agent are stored in the table Event Collector, also defined during model configuration. The Event Workspace is looped to select the applicable collector for the Event.
- Method -> Collect Event From the operational environment all events are retrieved which are stored in the ActualEvents that are filtered by the events in the Event Collector table and placed in a collection Event Workspace. This includes the actual value of the event. To select only the delta

events, select the events from the Actual Events table, which are stored for this simulation run and is agent independent.

Activity: Event Normalizer

After the events are retrieved from the operational context, they must be normalized. Normalization is necessary to define the context events in a format that can be recognized for further processing. This means that first the event normalizer set that is applicable will be selected. With this event normalizer, the collected events are translated into a known event. Another possibility is that the event is extended to complete the missing data. The third option is to change the format of the event into a recognizable format. According to Zhang *et al.* [35] the supervised ML technique would be supportive for normalization activities.

As the extension and formatting can be done by the transformation step, these are not applied in this model, so only translation is done.

- Method -> Get Event Normalizer: Select the normalizer that is applicable. This normalizer will be agent specific and is retrieved from the Agent Type, which is found in the Agent entity. In the Normalizer configuration table, the mapping between the agent type and the normalizer is defined. This normalizer will select the normalizer rules for each event for all records in the Event Workspace for the active agent. This workspace will be updated with the normalized events.
- Method -> Transform Event: In the Event Normalizer table, the target events are defined. These target events are created during configuration of the model. This function will transform the source event based on the Event Normalizer table mapping and will store the transformed events into the Event Workspace collection. For this model, the transformation has not been modelled.

Activity: Event Filter

The filter event will select only those event types that make sense for the agent to process. Only those events that are filtered out will be processed into symptoms.

- Method -> Get Event Filter: Retrieve the relevant filter for each event type for the agent (type). All events in the Event Workspace that need to be filtered will get a deletion indicator.
- Method -> Discard Event: Remove those events from the Event Workspace that have a deletion indicator.

Activity: Event Mask

When all events are filtered, the agent should consider if events should temporarily be neglected for a specific agent type. The reason for this is that events can be blacklisted, and not relevant is a specific time frame of net relevant for a context is relevant. In the Event Mask table, the events are marked with a mask status. This table is maintained in the ELV or CRP processes.

- Method -> Get Event Mask: Retrieve the event mask from the Mask table for this specific agent and set the deletion indicator in the Event Workspace.
- Method -> Ignore Event: Remove all events in the Event Workspace collection that have a deletion indicator.

Activity: Event Aggregator

All the events that are relevant could have duplicates. For analysis, it is not necessary to keep a single record for each event. For that reason, the duplicate records are removed from the Event Workspace.

- Method -> Merge Duplicates: Select all duplicate records and count them. Next remove these duplicate records and keep only one unique event in the Event Workspace.

Activity: Symptom Generator

The events in the Event Workspace collection must be translated into a generic symptom, stored in the Symptom table. This symptom characterizes a specific situation in the agent environment and is agnostic within this type of process, so it can serve several operational situations. The translation between events and symptoms is stored in the EventSymptom configuration table during the modeling of the managed system. The relevant Symptoms are updated in the Event Workspace.

- Method -> Infer Symptom: Select the events in the Event Workspace and retrieve the corresponding Symptom in the Event Symptom table. The resulting symptom is updated in the Event Workspace. Also, the parameter NoSymptom will get the value *True*. Otherwise, the default value *False* is set, which is the default value.

Gateway: Symptom Retrieved

In case the events have been collected, normalized, filtered, masked and aggregated and translated into Symptoms, the Symptom Workspace will have one or multiple records. In that case, this gateway will trigger the next activity (Send Symptom), defined by the NoSymptom parameter value *False*. However, when no symptoms have been retrieved, there is no further MAPE-K activity to be done. To trigger this exit, the “No Symptom Retrieved” end event is triggered by the NoSymptom value *True*. From here no feedback is sent to the managed system.

Event: No Symptom Sent

When no symptom sent has been triggered, this will result in this end event, and the main process is triggered by the next agent.

Activity: Send Symptom

As this model will use an agent specific common Symptom Workspace, it is not necessary to send the symptom to the Analysis process. So, for this approach the Send Symptom task is permanently skipped.

Event: Symptom Sent

This intermediate event will give a signal that a symptom has been released and can be picked up.

- Method -> Orchestrate Analysis: run the next process as defined in the BPMN monitoring process flow and trigger the Analysis phase.

Analyze

During analysis, the symptoms are analyzed, and contingency planning will be executed. This will result in a decision about how to proceed, described in a Request for Change (RfC) or triggering an end event to exit the flow when no change is required.

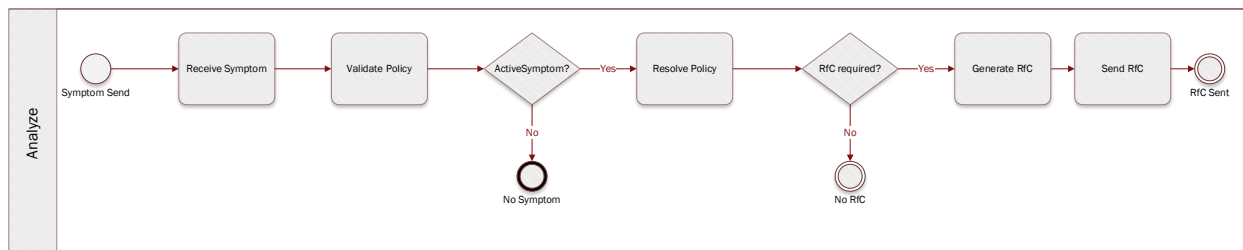


Figure 10 Analyze process

This process is orchestrated in the BPMN flow and digital twin by the orchestrateAnalysis method that coordinates the execution of the analysis process.

Event: Symptom Sent

In the Monitoring process, the symptom has been sent and will be picked up in this Analysis process.

Activity: Symptom Receiver

Check the Event Workspace for a specific agent. As the Analysis process is only triggered when there is a record in the Event Workspace, this process does not add any value and is skipped.

Activity: Policy Validator

The symptom will be checked if the symptom is active. In that case a policy could be applied. This is retrieved from the Symptom table. During the ELV process, this symptom can be created or (de)activated.

- Method -> Validate Policy: For each record in the Event Workspace of each agent, retrieve the symptom from the Symptom table and set parameter Active Symptom to *True* when the status is “Active”. By default, this parameter is set *False*.

Based on the symptom the relevant policy is triggered. This policy is stored in the SymptomPolicy table in the Knowledge Base.

This policy is the script to change the managed system and could imply different actions:

- Change the configuration of the managed system
 - Select or deselect a process pattern in the BPMS environment
 - Change a parameter in the execution layer
 - Activate or deactivate a business rule in the managed system
 - Trigger the execution of a process
 - Stop the execution of a process
 - (Re) Assign or remove a task to a role
 - Activate the execution of a function in the source code (script)
 - Or other execution behavioral change.
- Method -> Retrieve Policy: Retrieve policy from Symptom Policy table for the selected symptom in each Event Workspace record and store this in the RfC Workspace. This concerns the attribute Policy Id and Policy type. When no policy is retrieved, the NoPolicy parameter is set to *True*. As default this parameter has the value *False*. Next, this record will be removed from the Event Workspace.

Gateway: Active Symptom?

When the ActiveSymptom parameter has the value *True*, the process is continued. When *False*, the end event NoSymptom is triggered.

Event: NoSymptom

In this case the MAPE-K process will be stopped and returned to the Main process to hand over the process to another agent.

Activity: Resolve Policy

An applicable policy is defined as a Request for Change (RfC) and initially the symptom will only imply one RfC (but could be multiple in a sequence in the future). In this step the possible conflicts of policies are retrieved. When there are multiple RfC-s of the same policy type then the policy with the highest score will be applied. In addition, the expected results which are linked to the policy are stored. These expected results are defined in the policy table as double values and will be relative towards the standard.

The standard is defined manually upfront (or can be calibrated with simulation in the ELV cycle). The result of the applied policy will be retrieved from the managed system with the ActualResult variable. This generic variable has a property with the variable type. This variable type is assigned in the policy so it defines for which actual result the success analysis will be done.

To define the success, in the policy the Minimize or Maximize parameter is set. By default, the calculation is based on the minimizing function. In case of maximizing the outcome of the analysis it is reversed. In the method, a calculation is made by dividing the actual result by the standard

The outcome of this analysis will be a change in a parameter (policy change), activating or deactivating the process or doing nothing at all.

Method -> DetectConflict: Check if there are multiple records found in the RfC Workspace with the same PolicyType attribute. If positive the parameter RfCConflict is set to “True”, while its default is “False”

- Method -> Resolve Policy: When the parameter RfCConflict is “True” then retrieve the attribute *score* from the Policy table. Remove the records with the lowest scores for the same PolicyType and keep the one with the highest score.

Gateway: RfC Selected?

When the parameter NoPolicy is false, the process is continued and the next activity is triggered. When NoPolicy is True, the end event NoRfC is triggered. This record should be active, otherwise the next possible policy is selected. When there is no active policy set the *LearningRequired* indicator and trigger the ELV process.

Event: NoRFC

In this case the MAPE-K process will be stopped and returned to the Main process to hand over the process to another agent.

Activity: Generate RfC

The RfC in the RfCWorkspace will be enriched by the status, creator, creation timestamp and other relevant meta data. This data is used in the planning and execution process.

- Method -> Generate RfC: Add agent id from the runtime, add status “New” and creation timestamp (system date and time) to the RfC record in the RfCWorkspace. If needed, additional data can be added.

Activity: SendRfC

This activity is not relevant as the RfC workspace is used between processes to interface the RfC-s.

Event: RfC sent

This intermediate event will finish the Analysis process and will trigger the Planning process.

Method -> Orchestrate Analysis: run the Planning process as defined in the BPMN analysis process flow.

Plan

This subprocess will take care of the planning of the RfC in the managed system. Where and when will it be applied.’

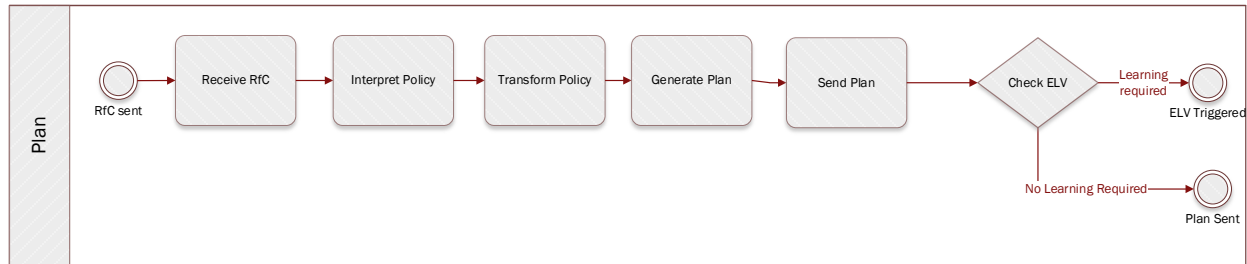


Figure 11 Planning process

This process is orchestrated by the BPMN flow definition.

Event: *RfC sent*

When the Planning process has entered a new RfC in the RfCWorkspace with status “New”, it will trigger this process.

Activity: *Receive RfC*

This activity takes care of picking a new RfC from the RfCWorkspace. When there are multiple RfC-s available, this process will be executed multiple times. For this model, only one RfC will be recorded in the Analysis process.

- Method -> *Receive RfC*: For each record in the *RfC Workspace* table, the process the status will be changed to “Planning”.

Activity: *Interpret Policy*

For each RfC there are some dependencies and specifications that are relevant before these can be applied. The RfC policy identifies these details and is used for planning. So, the interpretation is needed to translate the RfC in a plannable item. Changes will be defined as changes in policies or process patterns in the orchestrator, it is sufficient to trigger the process and transfer the parameters from the RfC to execute the process. When no policy details are available in the RfC Workspace, these will be added from the policy table.

- Method -> *Employ Policy*: read the *RfC Workspace* and for each RfC record with status “Planning” the corresponding policy is retrieved from the *Policy* table. Initially the corresponding record will give details on the policy type (like a configuration, process pattern, business rule, script, parameter, start or stop event, task assignment, function activation). Add these details to the RfCWorkspace in the RfC record. Also, the expected results are copied from the *Policy* table to *RfC Workspace*. This will be used during evaluation.

Activity: *Transform Policy*

This model is based on a BPMS execution architecture where the process orchestration is ruled by the BPMN convention. When another convention is applied, in this spot the transformation will be done. For this purpose, this step can be skipped.

Generate Plan

Create a list of execution scripts in the sequence defined in the RfCWorkspace. To achieve this, in this step the *PlanWorkspace* is created. Each record in the Plan Workspace will contain the execution script (to trigger the correct process pattern or activity parameter), the sequence indicator according to the sequence in which the actuation should take place. Initially, each record will have the status “Planning”. When this step is done the RfCWorkspace status of each RfC record is set to “Completed”.

- Method -> Generate Action Plan: For each record in the RfCWorkspace with status “Planning”, transfer the script to a new record in the PlanWorkspace table and set the status to “New”. Also, retrieve the object type, the attribute and value from the referring policy in the Policy table and store this in the Plan Workspace. Follow the sequence of each referring policy in the score attribute of the Policy table from low to high when the RfCWorkspace has multiple records. For this model, it will only contain one record. After committing the record, change the status of the RfC record in the RfC Workspace to “Completed”.

Gateway: Check ELV

When the system topology has been changed or if the learning has been activated randomly, the learning indicator is activated in the Main model. This indicator triggers the learning ELV process.

- Agent.vLearn = True:

Event: Plan ELKV

When Learning is required, the ELV process is triggered.

The orchestrate Execution method triggers the E:V process.

- Method -> orchestrate Execution: run the next process as defined in the BPMN analysis process flow

Event: Plan Sent

This intermediate event indicates that the plan to effectuate the RfC has been released. This means, that the PlanWorkspace has at least one plannable record.

The orchestrate Execution method triggers the next Execution process.¹

- Method -> orchestrate Execution: run the next process as defined in the BPMN analysis process flow

Execute

During the Execution process, the plan is effectuated into the managed system. When multiple actions should be executed, the process orchestration will be coordination the execution.

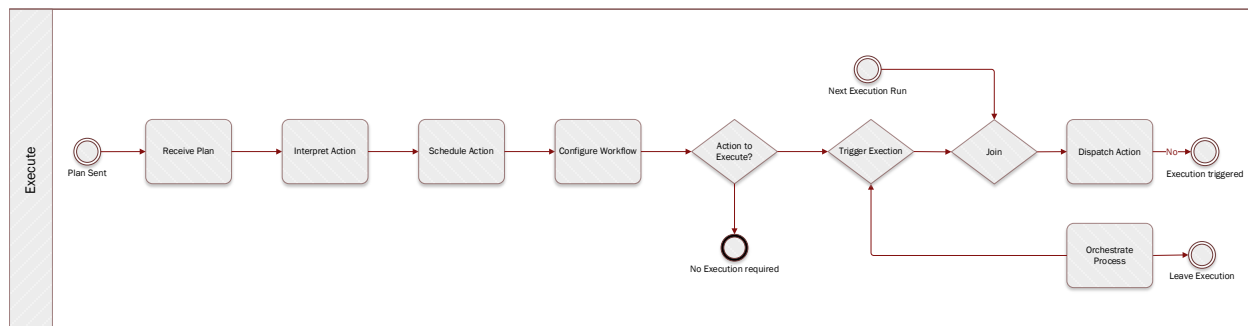


Figure 12 Execution process

¹ Orchestration in Anylogic is taken care of by the process modeling capability. When this would not be available, it could include the orchestration of all single steps.

This process is orchestrated in the BPMN flow.

Event: Plan Sent

When there is an action plan defined in the PlanWorkspace with the status “New”, it triggers the execution of this effectuator process.

Activity: Plan Receiver

As the action plan is listed in the PlanWorkspace, the forwarding and receiving of the planning message is not required, as the process is triggered with the Plan Sent event.

Activity: Action Interpreter

The action plan is stored in the PlanWorkspace in a script with storage of parameters. For this model only one process and decision policy records are stored. In this activity the process pattern is translated into an executable action. As the model is based on BPMN no further interpretation is needed, the BPMN engine will take care of the execution of a process applying the process pattern and related parameters. When another convention is used, the planning task Transform Policy will already take care of the conversion. That means that this activity of the Frameself methodology becomes redundant.

Activity: Action Scheduler

Based on the action plan the deployment of the action will take place. For each action the PlanWorkspace is updated with status “Scheduled”. In the orchestrator a time event will be activated on the date and time mentioned in the Action plan details. The Execution Workspace will be filled with the right sequence of process patterns and related decision policies. First, from the Plan Workspace the applicable process policies are retrieved. These are added in the right sequence to the execution workspace.

As processes are loosely coupled, for the first process policy in the Plan Workspace the preliminary process is retrieved. This is done by selecting that process from the Process Repository where the End Event equals the Start event (agent based). This process will continue until there is no preliminary process available. Next, the consecutive processes are retrieved by mapping the Start Event to the End Event of the process selected in the Planning Workspace. This will continue until there is no consecutive process found in the Process Repository table.

Finally, all processes are written to the Execution Workspace. Next, the related decision policies are retrieved from the Policy table, where the higher-level policy matches the process records in the Execution Workspace.

Method -> Schedule Action: For each record in the Plan Workspace add the date and time to this. Select the process start from the Plan Workspace for each record. Retrieve the preliminary process from the Process Repository table and continue until no other processes are found. Then do the same for the consecutive processes. After this, for all these selected processes, retrieve the related decision policies from the Policy table, based on the higher-level policy, which is the processed. Add these to the Execution Workspace as well. Set the status in the Plan Workspace to “Scheduled”. Also set the ExecuteAction parameter to “TRUE” (by default “FALSE”).

Gateway: Action to Execute?

This gateway checks if there is a scheduled action in the ExecutionWorkspace by reading the ExecuteAction parameter. If it has the value “TRUE” it will continue with the next activity. When “FALSE” the No Execution Required End event will be triggered.

Event: No Execution Required

This end event results in no further action and returns to the main process.

Gateway: Trigger Execution?

When there is a record in the Execution Workspace that has not been deployed to the operational environment, this will trigger the Action Dispatcher. Otherwise, all deployment has been done, and the MAPE-K loop can be finished.

- agent.ei < agent.executionWorkspace.size(): ei is the workspace index -> loop through all records.

Activity: Dispatch Action

When the action is scheduled, it should be deployed to the managed system. This is done by dispatching the script in the process engine. This process engine will take care of triggering the process intermediate event to activate the process pattern or deploying the parameter for the related process with the value stored in the PlanWorkspace. After this, the status of the record in the PlanWorkspace is set to “Sent”. When the API returns the result, this will be updated by “*Deployed*” or “*Error*” when failed.

Event: Execution Triggered

Trigger the Execution of the planned action.

When no remaining records in the Execution Workspace are available, the end event will be triggered.

For this fictive environment, a fictive process is triggered in the simulation environment.

- Method -> Dispatch Action: API trigger to the BPMS environment that includes the script from the Plan Workspace on the date and time that is scheduled. The return of the API will log if the dispatch of the process was successful in the BPMSLog table. When the parameter type and value is available in the PlanWorkspace record, this will be included in the API as well. When the return code is positive, the status will be set to “*Dispatched*”. When the API return code ends up in an error the status is set to “*Error*”. -> In this model, the processes are triggered to simulate the operational environment. So, no deployment to the BPMS is executed and no log is maintained.

Also, the parameter ConfigurationChanged will be set on agent level to process id when the script triggers a new process (identified as object type in the PlanWorkspace table. When the process is deactivated, the ConfigurationRemoved parameter will be set with the removed process id. These parameters will be used during the Evaluation process in ELV.

Activity: Orchestrate Process

This activity will be triggered when no further action needs to be done. Due to the behavior of Anylogic and the fact that execution is completely done in the simulated environment, this activity is passed to the end event Leave execution.

Event: Leave Execution

End state of the MAPE-K process and return to the main process.

The Orchestrate Process method coordinates the execution process.

Method -> Orchestrate Process: Trigger the main process and leave the MAPE-K cycle. The next agent will be activated.

Sub model MAPE-K learning capability

The learning process will improve the quality of the Knowledge Base by extending, (de-) activating knowledge and removing knowledge. This process consists of three steps: 1. Evaluate, 2. Learn and 3. Validate. This process is described clearly in the work of Kloes et al ([2].

For each ELV run, the CRP Exchange is triggered, to learn the latest insights for other agents.

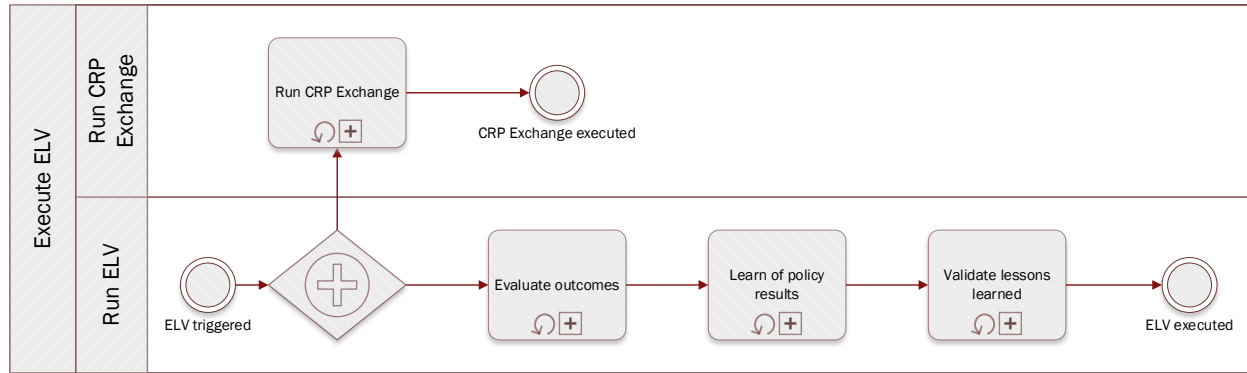


Figure 13 ELV process

Evaluate

In this step, the configuration of the managed system is checked, and the outcome of the previous execution run is evaluated. When the configuration has changed it will be checked if it has been deactivated. In that case, the corresponding policy will be deactivated as well. Also, the outcomes are evaluated against the expected policy outcomes. With this, as a result, the policy can be kept or replaced, indicated by the need to learn.

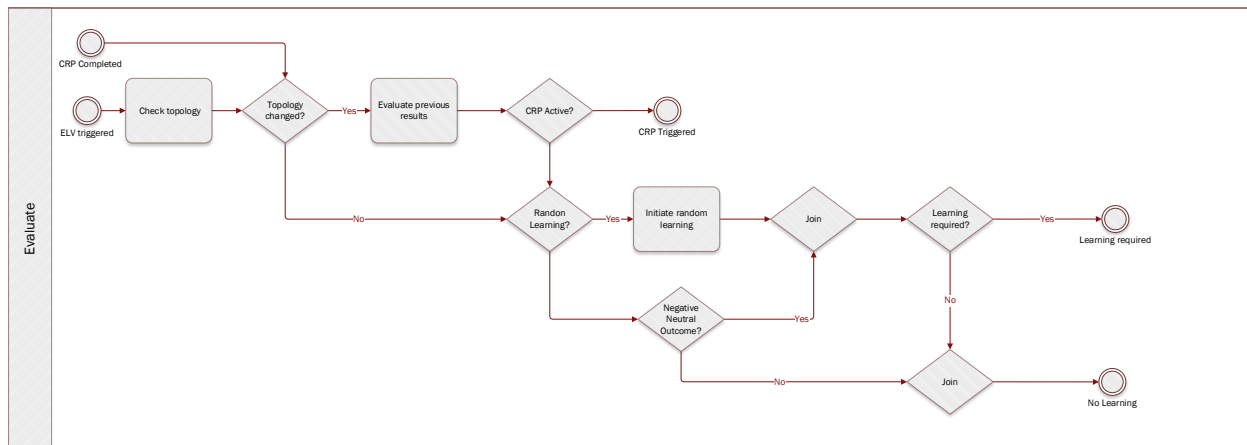


Figure 14 Evaluation process

This process is orchestrated in the BPMN flow.

Event: ELV Triggered

From the MAPE-K process, the ELV process is triggered. That implies that for each MAPE-K cycle a learning cycle will be executed. An indicator is set that this is the first time that the Learning cycle is run for this cycle.

Event: CRP Completed

When CRP is executed, it will return to the Evaluation process. In this cycle, it will check if there is a change of knowledge base (topology) to apply the new knowledge received from other agents.

- Method -> Add CRP Policy: When new policies have been received from other agents in the CRP process, add this policy to the Learning Workspace of the agent.

Activity: Check Configuration

When new policies were deployed to the operational environment, the configuration change workspace was filled with the relevant policies. When this agent specific collection has records, the learning workspace will be filled for each entry.

- Method -> Check Configuration: Loop through the agent collection Conf Changed and add each record to the Learning Workspace.

Gateway: Check Configuration

When the configuration has a new process pattern or the policy is changed, this will be indicated by the Learning Workspace has any records. In that case the answer is positive.

Activity: Evaluate Previous Results

The outcome of the process is evaluated against expectations. This is done by reading the events which came out of the Monitoring process and are stored in the Event Workspace. These results are compared with the expected results that are defined in the policy that is applied, which is found in the policy table. This expected result is stored as a double value in the policy table, for those records that are defined as a Process type. During learning, the policy gets an expected value of zero (0). During this evaluation step an algorithm is applied that compares the expected and actual results. This algorithm results in three possible values: *Positive* ($Expected > Actual$), *Negative* ($Expected < Actual$) and *Neutral* ($Expected = Actual$) and is stored in the EvaluationResult parameter. The precondition is that the results should be defined in such a way that a lower value means better. This is set in the policy table record as "Minimize" or "Maximize". When the EvaluationResult is "Positive", the expected value of the policy is overwritten.

When the policy and the process pattern originate from other agents, this is stored in the Policy and Process pattern table in the *Source* attribute. When the outcome of a policy or process pattern is Positive, the ranking of the Trust group is appraised with 100. For negative results, it will have a score of -100 and neutral is appraised by 0. These values are stored in the TrustGroupScore table, that includes the policy or process pattern id and the single score. When a new record is added, the new average score is calculated for all records in the Trust group score table for that Trust group and updated in the TrustGroup table. Because of this combination of value aggregation and policy iteration, this model can be qualified as the actor-critic *Generalized Policy Iteration* (GPI) technique [35].

Method -> Evaluate Results: Retrieve the value of the *ExpectedResult* for the Learning Workspace and the actual result from the Event Workspace. Compare both values. When $Expected < Actual$ then set the Need to Learn parameter (agent specific) to "Positive"; when $Expected > Actual$ then set the parameter to "Negative"; when $Expected = Actual$ then set the value to "Neutral".

When the actual result (when Learning is triggered from MAPE) or simulated result (triggered from simulation) is positive, nothing will be done. When neutral, finetuning is required by exploiting solutions. When negative, the exploration is triggered. The qualification positive, neutral or negative is based on the within a bandwidth around the expected value (positive and negative). This result (explore, exploit or none) will be stored in the learning method variable.

Also, calculate the average value of the weighted moving average Trust group score, stored in the *TrustGroupScore* table (attribute Average Policy Score) multiplied by the total number of scores and add value of the *SingleTrustScore* variable. Then divide by the total number +1 and overwrite this in the *AveragePolicyScore* attribute of the *TrustGroupScore* table for this specific policy. Also recalculate the new average score on the *Trust Group* table based on the average of the scores of all policies for this specific trust group.

Hence, for the determined outcome, update the *SingleTrustGroupScore* variable with the score: 100 when Positive, 0 when Neutral and -100 when Negative. When Positive, the learning method parameter is set as “None”. When Neutral it will be “Exploit”, when Negative it will be set as “Explore”. When the trigger is the initial MAPE cycle, it will compare the real process result from the operating environment. When it is triggered by a simulation, it will be compared with the simulation result.

Gateway: CRP Active

For each ELV cycle, the CRP run will be activated. However, as the Evaluation process will be executed after running the CRP, this gateway will prevent the process will run in a continuous loop.

Event: CRP Triggered

When CRP needs to be activated, the CRP process is triggered.

Gateway: Check Random Learning

When the configuration did not change and the learning method is empty or “random” this gateway is positive.

Activity: Initiate Random Learning

Select a random policy from the active policies that will be subject to learning

- Method -> *Random Learning*: Select from the *Policy* table for the active agent all decision policies that have the status “Active”. From this selection, choose a random policy and add this to the agents *Learning Workspace*.

Gateway Check Negative Neutral Outcome

When the outcome of the evaluation is negative, represented in the *Need To Learn* parameter, then the policy must be updated. This implies that the learning process will be triggered. Otherwise, the outcome is positive or neutral. In that case no need for learning is required. However, a random learning action could take place in the next activity.

Event: Learning Required

This intermediate event indicates that learning is required and will trigger the learning process by the *Orchestrate Evaluation* method that coordinates the execution of the Learning process.

Method -> *Orchestrate Evaluation*: run the Learning process as defined in the BPMN process flow.

Event: No Learning

This intermediate event indicates that no further learning is required and the process can be returned to the MAPE process.

Learn

This process focusses on learning out of internalized and externalized experience. Internalized experience is based on earlier outcomes of agent actions. Externalized experience is based on external data sources available for the agent (like databases, books, papers, research results and knowledge of other agents). The agent interaction will be further elaborated in the next chapter.

To be able to learn, a clear direction and objective is required. Guiding principles will channelize the learning trajectory, while clear objectives will direct the learning curve into a meaningful course. This objective is context specific, so unique for each single managed process. Therefore, during configuration of the managed process, the objective is clearly defined in Process Indicators and when possible parsed into its building blocks and dimensions.

Moreover, learning can only be done when the outcomes of the managed process are logged for analysis. When comparing the policy applied with the outcomes of the process, the conclusion can be drawn: the measure did positively contribute to the objectives, had a negative impact or did not have any significant contribution at all. With this outcome, the policy can be rated higher (positive result), lower (negative result) or equal (no significance). For this model only a negative outcome will result in the deactivation of a policy and a trigger to learn new policies. This is done in the evaluation process.

Also, during the learning step a sound balance between applying known knowledge, with only small variances within a small bandwidth (“exploitation”) and experimenting with new ideas (“exploration”) should be done. A third option is added, called “adaptation”, where existing (deactivated) policies are activated again.

There are many techniques available to manage this balance like the Bush-Mosteller RL model, the Roth-Erev RL model [36] or Q-Learning with its extensions [37]. At this point the emerging possibilities in the ML domain could contribute significantly but are left out of scope for this research.

The learning process consists of the following steps:

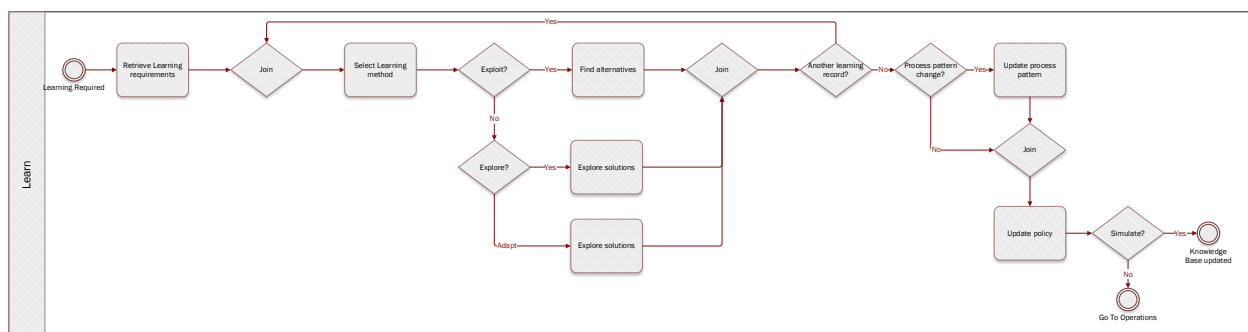


Figure 15 Learning process

This process is orchestrated in the BPMN flow.

Event: Learning Required

This event indicates that the Knowledge Base should be updated. This event will trigger the learning process.

Activity: Retrieve Learning Requirements

Retrieve the process pattern or policy that should be updated. Retrieval takes place from the *LearningWorkspace*. During Evaluation it can be realized that a process or policy should be updated. For

each record in the Learning Workspace, the Learning run should be executed. As this workspace is created in the Evaluation process, no specific activity is required.

- Method Retrieve Learning Requirements: Sort the Learning Workspace based the policy type.

Activity: Select Learning Method

This step is used to find out how the learning will take place. For each instance it is decided whether Exploration or Exploitation will be applied and retrieved from the Learning Workspace. When the current knowledge is exploited, then the existing process patterns or policies are chosen. For exploration, new knowledge is created or gathered. To answer this question, we can rely on many techniques in the ML RL domain. For this purpose, the Q-Learning technique [37] is used, where the learning from the past is translated into policy scores to indicate the success of the applied policy. The Q-Learning technique is model-free and easy to implement. This technique is defined during configuration of the model and stored in the LearningTechnique table. For Q learning the balance between Exploration and Exploitation is stored in the system tables and retrieved.

Method -> SelectLearningMethod: From the configuration tables retrieve the learning technique, which is stored in the LearningTechnique table and place this in the AppliedLearningTechnique parameter for Balance, ML Exploration and ML Exploitation. Hence, retrieve the learning method from the Learning Workspace, which is used for this learning cycle.

Gateway: Exploit?

In this gateway the direction for further exploitation or exploration is given. This will be driven by the AppliedLearningTechnique. When this parameter has the value “*Exploit*” it will trigger the exploitation flow. Otherwise, it will drive the exploration flow.

Gateway: Explore?

In this gateway the direction for further exploration or otherwise adaptation is given. This will be driven by the AppliedLearningTechnique. When this parameter has the value “*Explore*” it will trigger the exploration flow. Otherwise, it will drive the adaptation flow.

Activity: Exploit Solutions

When the performance of the system delivers reasonable results, but could be improved, this activity of Exploitation will generate small differences. For this, the current policy value is changed slightly, by adding or lowering the existing value with 1 integer.

- Method: Exploit Solutions: Retrieve the tolerance level from the DB Learning Technique table for this agent. Then find policy results from the Actual Outcome tables and define the bandwidth. For each policy in the learning workspace that is qualified as Exploitation, add or decrease the value with 1. This value is added to the Learning Workspace. In the Learning Workspace, fill the Previous Policy to be able to restore the value after learning.

Activity: Explore Solutions

When the current knowledge is not sufficient, the agent will explore new solutions. This could be the composition of new processes or the application of changed parameters. This exploration is the domain of many techniques which will be chosen earlier in the process. At this spot the learning activity is executed in a specific algorithm or via an API to an AI platform. The outcome of this learning will be a new process pattern, which is created by the algorithm or API as an XPDL file, which will be converted as a BPMN diagram in the BPMS platform. This new process pattern id will hence be stored in the Process Pattern table. Another outcome will be a change of RfC policy. The new process patterns first are stored in the Learning Workspace (by its new process id and new policy id). For this, the policy type is selected

by combining with the object type and attribute. For this combination a new random value is selected. However, to find an appropriate value, the range (lowest and highest value) is retrieved from the policy configuration table, and a value is selected randomly.

- Method -> Explore Solutions: Apply the learning technique which is stored in the AppliedLearningTechnique parameter. When Machine Learning is active, trigger the ML interface. Also, run the algorithm which is stored in the Learning Technique table as a script or run an external learning service like AI with an API. This will result in an XPDL script and a policy script. Next, run an API to the BPMS platform to convert the XPDL into a new process pattern. Then receive the process id and store this in the Process Pattern table, including the name of the process, XPDL name, source and class. The source is the agent's name when the algorithm is applied. When an API is used includes its source. The class is copied from the original process id. Then update the Learning Workspace with the new ProcessPattern_Id. In the Learning Workspace, fill the Previous Policy to be able to restore the value after learning.

The new policy is created by running the policy algorithm or the API which is stored in the Learning Technique table. The results are stored in the Policy table and updated in the Learning Workspace. The id, source and class will be retrieved equal to the process pattern.

For this process change the parameter NewProcess is set to "TRUE" or for a new policy the parameter NewPolicy is set to "TRUE"

Activity: Find Alternatives

To exploit the available knowledge in the Knowledge Base, alternatives are searched with the same process pattern type. To achieve this, process patterns are classified for their usage. Within the ProcessPattern repository in the BPMS, each activity of subprocess is classified. All these process patterns that belong to the same classification and have status "Inactive" can be used as alternatives. These alternatives are identified by its process id and listed in the Learning Workspace. For policies, this same classification can be used to derive alternative policies in the Policy table, as policies are linked with each process pattern.

- Method -> Find Alternatives: For each process id search for alternative process id-s within the same classification in the ProcessPattern table. Then select a random alternative and store this in the Learning Workspace in the newProcessPattern_Id attribute. The parameter ProcessChanged is set to "TRUE". When Machine Learning is active (policy learning configuration table), this ML interface is triggered. Also, each process pattern retrieves its policy. Next search for alternatives with the same classification as the original policy. Also, choose a random policy from the list of possibilities where the status is "Inactive" and store this in the Learning Workspace in the newPolicy_Id value. Mark the status of this record to "Changed". The parameter PolicyChanged is set to "TRUE". In the Learning Workspace, fill the Previous Policy to be able to restore the value after learning.

Gateway: Another learning record?

When there is a next record in the Learning Workspace then the next learning loop is executed, and a new learning method is selected.

Gateway: Process Pattern change?

When there is a change or a new process pattern in the LearningWorkspace, the process flow in the BPMS must be updated. Otherwise, the next gateway check is called. When the process pattern is changed, it

could also be the case that also the policy is changed. For this reason, this gateway is a complex gateway. This is indicated by the parameter NewProcess or ProcessChanged with value “TRUE”.

Gateway: Policy change?

When this is a new or change of policy in the Policy Workspace, identified by the parameter PolicyChanged is set to “TRUE” or parameter NewPolicy or PolicyChanged is set to “TRUE”, the policy change will be applied. Otherwise, the change will be skipped.

Activity: Update Process Pattern

When the process pattern needs to be changed, the process pattern, defined as a BPMN model will be changed and activated in the BPMS platform.

- Method -> Update Process Pattern: Retrieve the process id in the LearningWorkspace in attribute newProcessPattern_Id and call the BPMS platform to activate the process pattern. To find the right API and process id, retrieve this from the ProcessPattern table. Also update the ProcessPattern table in the Knowledge Base. When the NewProcess parameter is “True” add a new record to the ProcessPattern table. When the ProcessChanged parameter is “True” retrieve the corresponding record from the ProcessPattern table and deactivate the process pattern details with the corresponding attributes in the Learning Workspace.

Activity: Update policy

When the policy has changed, it should be applied in the Knowledge Base. The existing Policy will never be overwritten in case of change, only the status could be changed. When there is a new record, it will be added to the Policy table.

- Method -> Update Policy: The RfCPolicy will be changed when this has been indicated in the NewPolicy or PolicyChanged parameter is “True”. When the NewPolicy is active, add a new record to the Policy table, with the attributes from the LearningWorkspace. For the PolicyChanged parameter is “True” retrieve the policy id from the Policy table and deactivate all policies. Also create a new record in the SymptomPolicy table, where the Symptom is copied from the original policy (this will not be done in the initial phase of the project).

Gateway: Simulate?

When simulation of the new policies is required, the indicator Check Simulate is activated. This will trigger the Validation process.

Event: Knowledge Base updated.

After this process the Knowledge Base has been updated, and the Validation process is triggered.

Validate

After the new or updated knowledge is added in the Knowledge Base, it must be validated. Validation implies a meta simulation of the process and check if the outcomes are better than the earlier results and updates the policy expectation. When it fails, the process pattern and/or policy are deactivated.

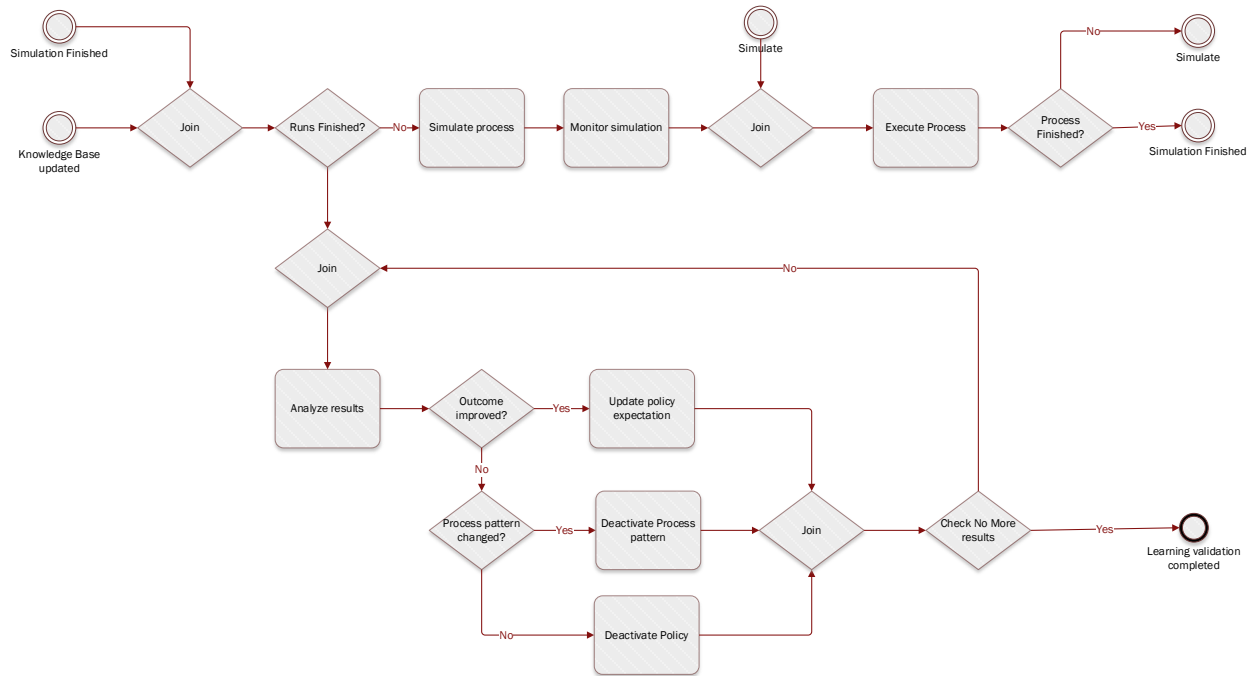


Figure 16 Validation Process

This process is orchestrated in the BPMN flow.

Event: Knowledge Base updated

When the Learning process has updated the Knowledge Base, it is ready to be validated. This event will trigger the validation process and reset the simulation parameters before it is filling the new simulation workspace.

- Method -> Reset Sim Parameters: Remove all values from the agent simulation variables.
- Method -> Fill Simulation Workspace All: Fill the Simulation Workspace for all processes in the Learning Workspace

Event: Simulation Finished

When the simulation of one workspace has been completed, the simulation parameters are restored before the next record is executed. This is possible because the simulation results have been stored in the simulation results table.

- Method -> Reset Sim Parameters: Remove all values from the agent simulation variables.

Gateway: Runs Finished

When all records in the simulation workspace have been processed, the analysis process is triggered.

Activity: Simulate Process

Run the new or changed processes from the Simulation Workspace in simulation mode. For this purpose, the simulation of the managed process is triggered, and the MAPE-K is applied. So, the managed system will not be impacted as the only objective of this step is to validate the lessons learned.

Method -> Simulate Process: Select the simulation of the managed process in the simulation environment for this specific agent in the Simulation Workspace Init.

Activity: Monitor Simulation

Select the process to simulate in the Simulation Workspace Init and fill the final Simulation Workspace with the full process sequence that is retrieved from the Process Repository tables, identical to the logic defined in the Planning process of MAPE.

Method -> Retrieve Sim Results: Retrieve the process to be simulated from the Simulation Workspace Init, retrieve the preliminary and consecutive processes based on start and end events in the Process Repository. Then, store this in the Simulation Workspace.

Activity: Execute Process

In Anylogic the effector for a sub process is placed at an exit object. For that reason, this activity will only maintain the counter to indicate what process leg has been executed.

Gateway: Process Finished

When all processes for this simulation record are completed, it will go to the next record. In that case the results are written to the database.

- Method -> Add to Log: Add simulation results for this record to the Simulation Workspace Log.

Event: To Simulate

When there is a next process leg to be executed, this will be triggered in simulation mode. This triggers the main process and includes the execution of updated process patterns and policy. Exclude the ELV and CRP processes but include the MAPE-K loop to understand the overall result

- Method -> Get Agent Sim Results: Select the variables that are needed to execute the process from the agent database.
- Method -> Execute Process: Execute the simulation of the operational processes from the Simulation Workspace in the right sequence and include the variables and parameters found in the earlier simulation and learning cycle.

Event: Simulation Finished

When all processes of the simulation record have been completed, it will trigger the next run.

Activity: Analyze results

Analyze the results in the SimulationWorkspace for each outcome record. First calculate the simulated totals (like the total costs in the Beer Game). Hence compare this with the process pattern and policy expectations of the original process pattern and policy. When the outcome is equal (neutral) or lower (positive), the updated policy is kept in the Knowledge Base. When the outcome is higher (negative) than the expectation, the changed process patterns and policies are deactivated.

- Method -> Calculate Total Results: Calculate the results for the processes indicators (here from Beer Game Sim).
- Method -> Analyze Results: For all process patterns and policies in the Simulation Workspace retrieve the expected outcomes from the expected result value in the Simulation Workspace record. Then compare the actual and the expected result for each outcome type. The parameter OutcomeImproved will be set to “TRUE” when the actual outcome is equal or lower than expected. Otherwise, it is kept to its default value “FALSE”.

Gateway: Outcome improved

When the parameter OutcomeImproved is “TRUE” than the policy and process patterns are kept as it is.

Gateway: Process pattern changed

When the parameter NewProcess or ProcessChanged is “TRUE”, then the Process change must be rolled back. If “FALSE”, the policy change will be rolled back.

Activity: Update policy expectation

When the outcome is equal or improved the expectation of each policy should be updated. For this purpose, the changed or new policies are retrieved from the Simulation Workspace and the expectation of each Policy is updated in the Policy table and the Policy Score table in the Knowledge Base with the actual value of the outcomes.

- Method -> Update Policy: For each policy in the SimulationWorkspace, retrieve the expectation attribute in the Policy table and replace the value with the actual outcome from the Simulation Workspace. Also, retrieve the trust group for this agent and policy from the Trust Group Agent and Policy Score table, calculate the new average score and update the Policy Score table for that policy and trust group.

Activity: Deactive Process pattern

Retrieve the process ID from the SimulationWorkspace. Then, call the BPMS API and deactivate the new process id-s that have status New and roll back to the previous version of the process id that has been changed in the Process pattern table.

- Method->Deactivate Process: When the NewProcess parameter is active, then deactivate the process. Retrieve the process id from the Simulation Workspace and search for the process id and API in the Process Pattern table. Then send the deactivation call to the BMPS. Also change the status of the process pattern to “inactive”. When the Process Changed parameter is active, search for the process_id and API in the ProcessPattern table. Then send the version roll back call to the BPMS to activate the previous version and activate the previousprocess pattern tablerecord.

Activity: Deactivate Process

Retrieve the process policy from the Simulation Workspace. For new process policies, in the Simulation Workspace, deactivate the policy. When the policy has status Changed, activate the previous value of the policy.

- Method->Deactivate Process: Retrieve the policy id from the Simulation Workspace where the type is Process. When the _policy status in this record is active, then change the status of the corresponding policy in the Policy table into “inactive”. When the status is “Changed”, change the status of the corresponding policy in the Policy table into “inactive” and retrieve the previous policy version from the inactive policy record and make this active by changing the status to active. Also, retrieve the corresponding entry in the Symptom Policy table and deactivate this record.

Activity: Deactivate policy

Retrieve the decision policy from the Simulation Workspace. For new decision policies, in the Simulation Workspace, deactivate the policy. When the policy has status Changed, activate the previous value of the policy.

- Method->Deactive Process: Retrieve the policy id from the Simulation Workspace where the type is Decision. When the _policy status in this record is active, then change the status of the

corresponding policy in the *Policy* table into “inactive”. When the status is “Changed”, change the status of the corresponding policy in the *Policy* table into “inactive” and retrieve the previous policy version from the inactive policy record and make this active by changing the status to active. Also, retrieve the corresponding entry in the *Symptom_Policy* table and deactivate this record.

Gateway : Check no more Results

When all records in the Simulation Workspace have been analyzed and actions are taken, the Validation process will stop. Otherwise, the analysis step for the next record in the *Simulation Workspace* is triggered.

Event: Learning validation completed

This result indicates that the learning cycle has been completed and return to the Evaluation process This is achieved by executing the *Orchestrate Validation* method.

- Method -> *Orchestrate Validation*: trigger the Evaluation process as defined in the BPMN Validation process flow.

Sub model CRP extension

With the MAPE-K ELV control loop each agent can improve the execution of the managed system by changes in configuration or business rules. The agent’s Knowledge Base is, however, internalized and not open for external input. The CRP extension will take care of this piece where knowledge is exchanged with other agents, organized in trust groups. This exchange is bi-directional, whereas each agent is sharing its knowledge but also retrieves knowledge from peers. Because of this, the MAPE-K control loop becomes a genuine *social collaborative system*. As the triggering of the CRP is done before the learning cycle takes place, each agent can use the latest knowledge in their learning process. However, the agent can apply new knowledge first before it is shared with other agents in the next cycle.

Gesture CRP Exchange

The Gesture process is triggered for each ELV run (if this indicator is activated in the start page of the simulation). The active agent will send a policy request and sends its own best scoring policy. Also, it could be possible that there are already requests from other agents in its workspace, which will be used for this. Sharing of knowledge will only be done within its own trust groups.

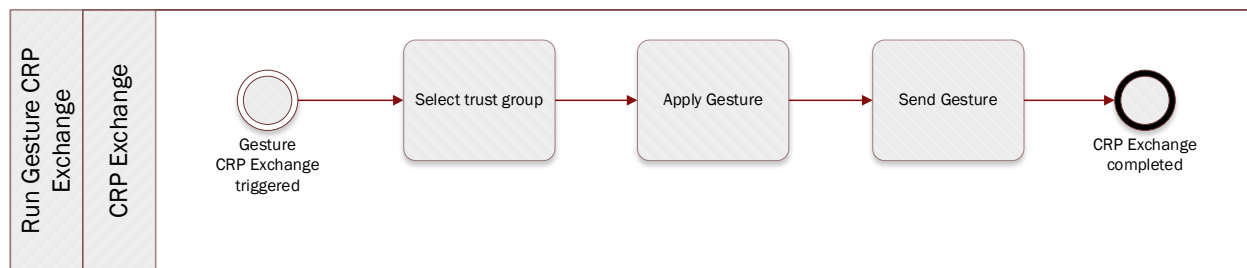


Figure 17 Gesture CRP Exchange

Event: Gesture CRP Exchange triggered

This intermediate event is triggering the CRP exchange process and originates from the start of the ELV process. In this step, the CRP Workspace will be filled with the initial Gesture that will be processed.

- Method: Fill CRP Workspace: For all records in the learning workspace, the records will be moved to the CRP Workspace.

Activity: Select Trust Group

For each record in the CRP Workspace, the corresponding trust group is retrieved.

- Method -> Select Trust Group: Loop through the CRP Workspace. For the object type and attribute of each policy id found, the related trust group is retrieved from the Trustgroup table and stored in the Selected Trustgroup workspace for this agent.

Activity: Define Gesture

For all records in the CRP Workspace, select the best performing policy (highest score) and place this in the Gesture List for each agent that is part of the Selected Trustgroup.

- Method -> Define Gesture: Loop through the CRP Workspace, retrieve the policy with the highest score for the specific object type and attribute and add this to the Gesture List workspace for this agent.

Activity: Send Gesture

For all records in the Gesture List the agents are selected as part of the selected trust group. For this agents, the gesture (policy request) is added to the CRP Workspace of all these agents.

- Method->Send Gesture: For each Gesture List record, retrieve the agent that is linked to the Selected Trust Groups from the Trust Group Agent table. Then add these records to the CRP Workspace of all these agents by looping through the Subject workspaces.

Event: CRP Exchange completed

This end event indicates that the CRP Exchange has been completed. In this step, the Response process is triggered.

Response CRP Exchange

The Response process is triggered when there is a request for knowledge from other agents. First the received knowledge (policy) is stored in its own knowledge base. Then, the agent will send its own best scoring policy for that object type and attribute to the other agents as a response.

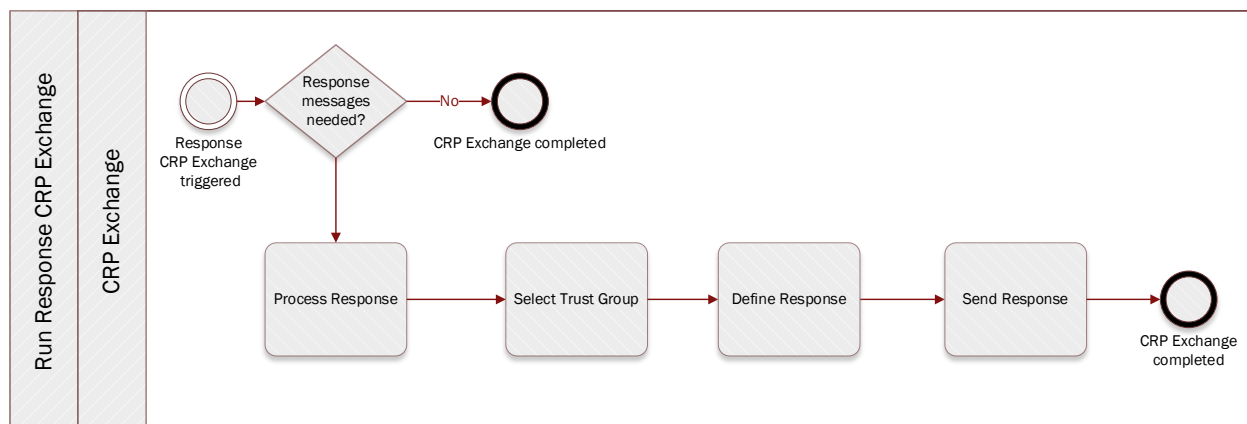


Figure 18 Response CRP Exchange

Event: Response CRP Exchange triggered

The Gesture CRP process will trigger the Response CRP process. This event will start the inbound exchange procedure.

Gateway: Response messages needed?

When the CRP Workspace contains records, it means that messages should be retrieved. So, in this flow these will be retrieved and so the process is started. If not, the end event is triggered.

Activity: Process Response

When the agent must give a response to requested knowledge (status “Requested”), it will be based on the CRP Workspace record. The requested policy will first be stored in the agent’s knowledge base. For this the policy in the record in the CRP Workspace is retrieved from the Policy table. As policies are not harmonized, it will retrieve the policy characteristics from a known policy based on the object type and attribute (or kept empty when no reference can be found). This policy is added to the Policy table and the Symptom Policy table. Hence, the status of the CRP Workspace record status is updated as “Processed”.

- Method -> Process Response: Loop through the CRP Workspace and select the records where the source is another agent and the status is “Requested”. For this record, retrieve a reference policy from the Policy table based on the object type and attribute from the CRP Workspace record. Copy the properties of this policy to the new policy when the request details are missing. When no reference policy is found, keep these values empty. Also, add the policy to the Symptom Policy table based on the reference policy.

For process patterns, do the same on the Process Pattern table. When that is done, deploy the process XPDL script, which is stored in the message attribute, to the BPMS platform to create a new process pattern as BPMN file. This is out of scope for this study.

Activity: Select Trust Group

For each record in the CRP Workspace, the corresponding trust group is retrieved. In this way, new policies are distributed quickly between agents in Trust groups.

- Method -> Select Trust Group: Loop through the CRP Workspace. For the object type and attribute of each policy id found, the related trust group is retrieved from the Trustgroup table and stored in the Selected Trustgroup workspace for this agent.

Activity: Define Response

For all records in the CRP Workspace, select the best performing policy (highest score) and place this on the Response List for each agent that is part of the Selected Trustgroup.

- Method -> Define Gesture: Loop through the CRP Workspace, retrieve the policy with the highest score for the specific object type and attribute and add this to the Response List workspace for this agent.

Activity: Send Response

For all records in the Response List the agents are selected as part of the selected trust group. For this agents, the gesture (policy request) is added to the CRP Workspace of all these agents.

- Method->Send Response: For each Response List record, retrieve the agent that is linked to the Selected Trust Groups from the Trust Group Agent table. Then add these records to the CRP Workspace of all these agents by looping through the Subject workspaces.

Event: Response Out

Intermediate event that indicates that the requested message in the *CRP Workspace* is processed and policies are shared between agents.

Sub-model Operational Environment

In this study, the Beer Game is used as an operational model. This means that the processes are modeled in Anylogic in a similar way to the original model. As a baseline, the model of Edali and Yasarcan is used. This model identifies the four echelons, and the model is based on a temporal space dimension with a discrete cycle time of one week. More details can be found in the study of Edali and Yasarcan.

The number of agent could vary in the MAPE-K ELV CRP model, however, as the baseline model only contains the four echelons, the number of agents is fixed to four (and a customer for independent demand).

Each process is modelled as a separate agent that represents the process, both in the operational and simulation workspace.

Operations Workspace

bg_RegisterSalesOrder bg_ReceiveGoods bg_ProduceGoods bg_DeliverGoods bg_CalculateStock bg_CreatePurchaseOrder



Simulation Workspace

sim_RegisterSalesOrder sim_ReceiveGoods sim_ProduceGoods sim_DeliverGoods sim_CalculateStock sim_CreatePurchaseOrder



The processes are detailed for each agent type in the process repository as follows:

process_id	policy_id	process_name	process_type	role	start_event	end_event	event_type	in_scope	sequence
1	4	Register Sales Order	Service	Sales Manager	E1	E2	ET1	☒	1
2	5	Receive Goods	Service	Warehouse Manager	E2	E3	ET2	☒	2
3	6	Produce Goods	Service	Production Manager	E3	E4	ET3	☐	3
4	7	Deliver Goods	Service	Warehouse Manager	E4	E5	ET4	☒	4
5	8	Run MRP	Service	Planner	E5	E6	ET5	☒	5
6	9	Create Purchase Order	Service	Purchaser	E6	E7	ET6	☒	6

For the manufacturer, the Produce Goods process is set in scope.

In these processes, the variables and parameters are used to calculate the results and drive the dynamics of the model. This is a simulation of a four-echelon supply chain, modelling inventory, orders, and costs over time, with dynamic adjustments based on demand forecasts and stock levels. The goal is likely to analyze the impact of inventory policies on total costs.

Logical Description of Beer Game Simulation coding

This model implements a **multi-period inventory management simulation**, representing a supply chain with four echelons (Retailer, Wholesaler, Distributor, Factory). The model simulates the flow of goods,

orders, and inventory levels over 36 periods ($t = 1$ to 36 in the mathematical model, but $t = 2$ to 37 in R due to indexing).

1. Variable Initialization

- **Dummy Variables:** Many variables are initialized to zero. These will store values for inventory, orders, backlogs, and costs over time.
- **Parameters:** Fixed values for the model, such as:
 - sat_* : Adjustment speeds for inventory.
 - mdt_* : Lead times.
 - st_* : Safety times.
 - plt : Production lead time.
 - wsl_* : Weighting factors for inventory adjustments.
 - theta_* : Smoothing parameters for expectation formation.

2. Initial Conditions

Demand and Order Data:

- $\text{ended}[t]$: End consumer demand, set to 4 for $t = 1-4$ and 8 for $t = 5-36$.
- Initial expectations (eecd , eo_*) and orders (o_* , io_* , psr) are set to 4.
- Initial inventory (i_*) is set to 12 for all echelons.

3. Simulation Loop ($t = 2$ to 37)

For each period, the following steps are executed:

Step 1: Total Inventory Calculation

- $\text{ti}_*[t]$: Total inventory (on-hand + in-transit) for each echelon.
- $\text{iti2}_*[t]$ and $\text{wipi2}[t]$: Intermediate inventory variables, updated from previous period.

Step 2: Sales and Backlog Update

- $\text{s_endc}[t]$, $\text{s_r}[t]$, $\text{s_w}[t]$, $\text{s_d}[t]$: Sales are the minimum of total inventory and demand (or available stock).
- $\text{iti1}_*[t]$ and $\text{wipi1}[t]$: Intermediate inventory variables, updated based on sales.

Step 3: Backlog and On-Hand Inventory Update

- $\text{b}_*[t]$: Backlog (unmet demand) for each echelon.
- $\text{i}_*[t]$: On-hand inventory after sales.

Expectation Formation

- $\text{eecd}[t]$, $\text{eo}_*[t]$: Updated expectations for demand and orders using exponential smoothing (adjustment based on theta_*).

Step 4: Order Propagation

- $\text{io}_*[t+1]$: Orders placed by downstream echelons become incoming orders for upstream echelons.

Step 5: Desired Stock Level and Order Adjustment

- `dsl_[t]`: Desired stock level, calculated using updated expectations and lead times.
- `ei_[t]`: Excess inventory (on-hand minus backlog).
- `sl_[t]`: Stock level (on-hand + in-transit + intermediate inventory).
- `sla_[t]`: Stock level adjustment, based on the gap between desired and actual stock.
- `ia_[t]`: Inventory adjustment, based on demand and excess inventory.
- Order Decision:
 - For $t \leq 5$, orders are fixed at 4.
 - For $t > 5$, orders are calculated as: $\max(\text{expected_demand} + \text{inventory_adjustment} + \text{stock_level_adjustment}, 0)$ and rounded using $\text{floor}(\max(\dots, 0) + 0.5)$ to mimic "round halfway from zero."

Cost Calculation

- `tc_[t]`: Cumulative holding and backlog costs for each echelon, updated each period.

4. Output

- **Total Costs:** The script outputs the total cost for each echelon at the end of the simulation (`tc_[37]`).
- **Team Total Cost:** Sum of all echelons' costs (`ttc`).

Key Features

- **Supply Chain Dynamics:** The model captures the bullwhip effect, where demand variability amplifies upstream.
- **Inventory Management:** Uses exponential smoothing for demand forecasting and dynamic stock level adjustments.
- **Cost Calculation:** Holding and backlog costs are tracked for each echelon.

Mathematical Model Mapping

- The code maps directly to a mathematical model, with comments referencing equation numbers (e.g., #1, #2, etc.).
- The loop runs from $t = 2$ to $t = 37$ in R, corresponding to $t = 1$ to $t = 36$ in the mathematical model.